

多旋翼飞行器设计与控制
系列实验

RflySim高级版 第7讲 集群控制算法开发

戴训华 daixunhua@163.com
中南大学 计算机学院 副教授
北京航空航天大学 自动化学院 博士



北航可靠飞行控制研究组
BUAA Reliable Flight Control Group

大纲

1. 安装配置
2. 接口介绍
3. 基础例子
4. 进阶例子
5. 小结

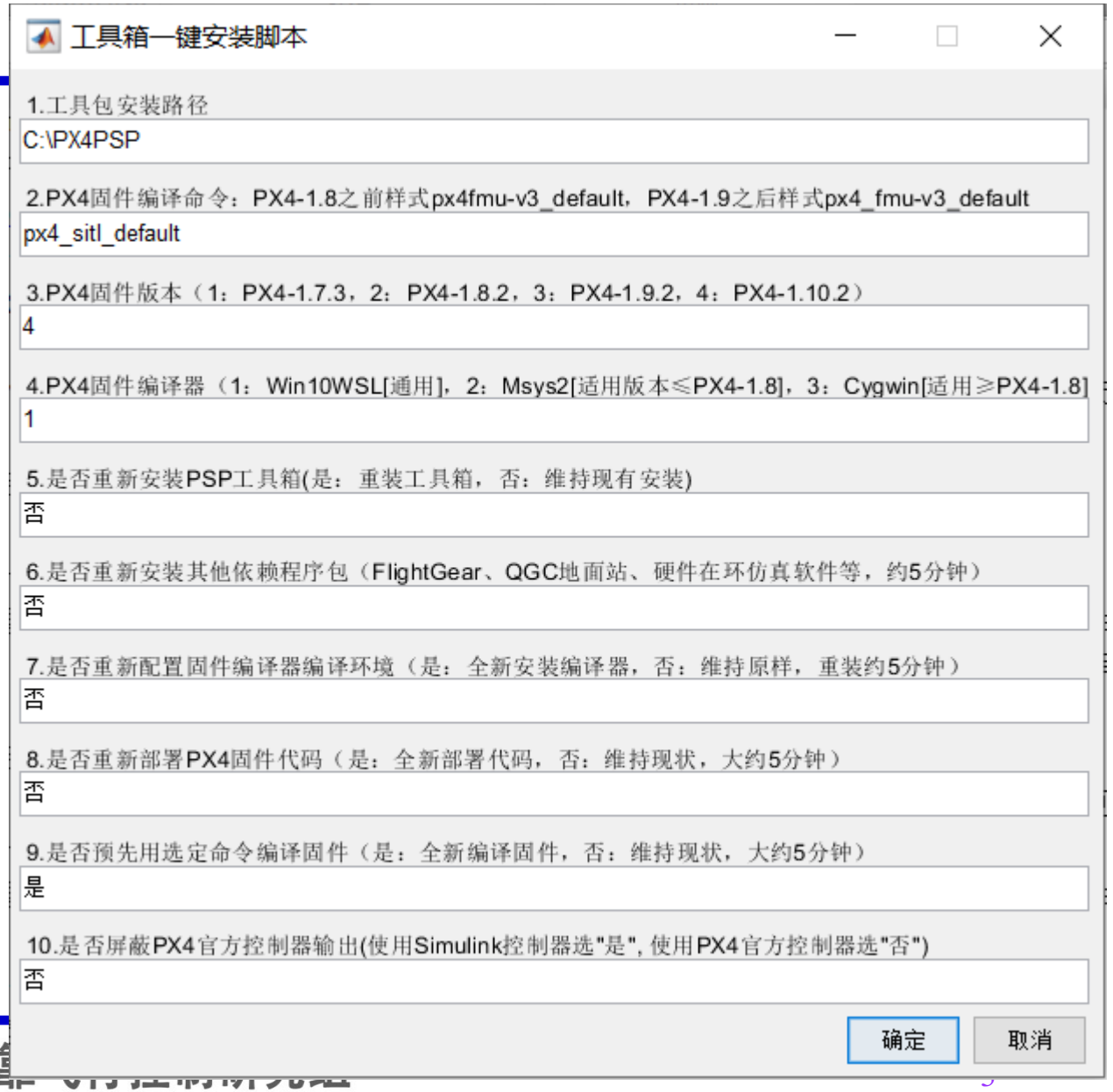
本讲的例程源码路径：
RflySimAPIs\SimulinkSwarmAPI

1.安装配置

1.1 RflySim平台配置

- 本平台用于集群算法开发时，推荐重新运行安装包内“OnekeyScript.p”脚本，并使用右图配置运行：

1. 使用PX4 SITL软件在环固件编译命令输入“px4_sitl_default”
2. 使用最新的PX4固件 PX4-1.10.2，固件版本选择“4”
3. 使用Win10WSL编译器，因此编译器选“1”
4. 是否屏蔽PX4输出项目选择“否”
5. 点击“确定”按钮即可开始安装。



如果要使用Pixhawk官方控制器进行硬件在环仿真，还需要按照“RflySim高级版_第01讲_总体介绍.pptx”中2.5节方法通过QGC正确配置Pixhawk自驾仪

1.安装配置

本平台需要使用Visual Studio 2015、2017或以上的C++编译器（不支持MinGW等），因此需要提前安装Visual Studio并配置MATLAB进行识别。请按照高级版课程第1讲的1.8小节步骤安装VS编译器

1.2 MATLAB的VS C++编译器配置确认

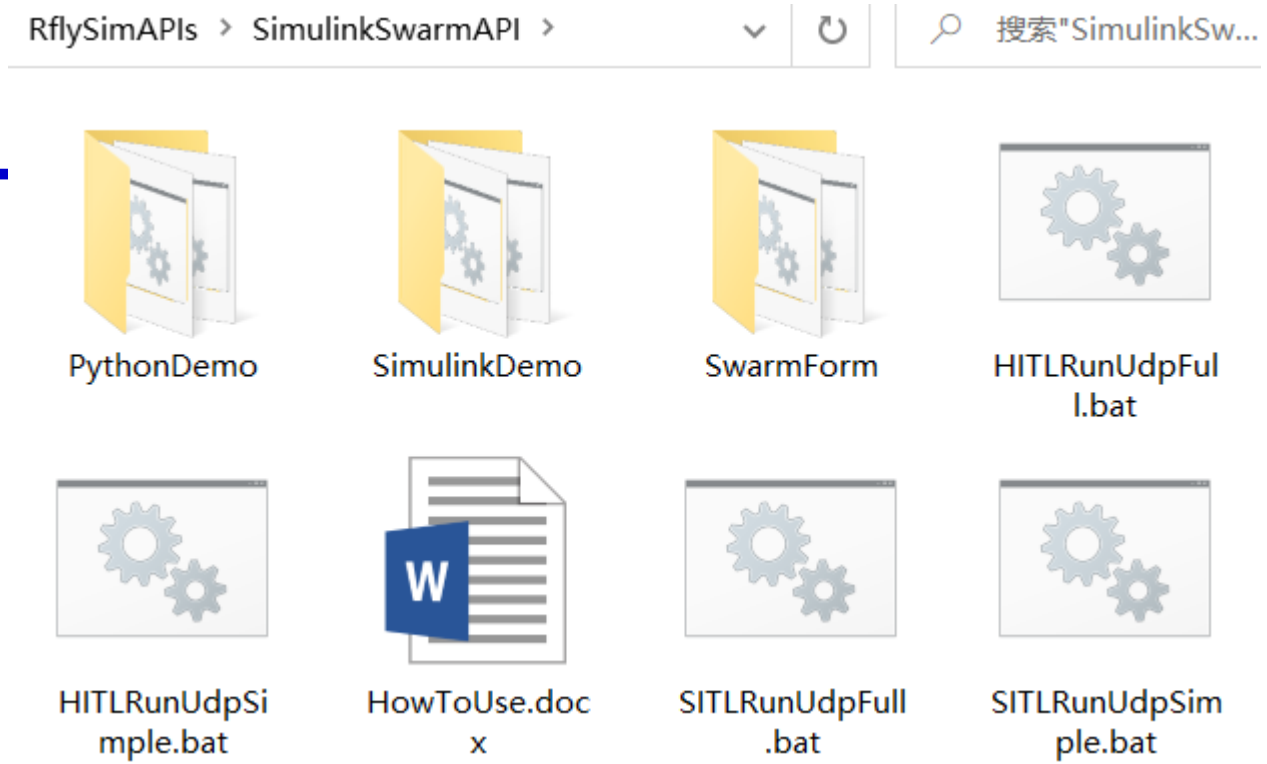
- 在MATLAB的命令行窗口中输入指令“mex -setup”
- 一般来说会自动识别并安装上VS 编译器，如右图所示显示“MEX配置使用‘Microsoft Visual C++ ***’以进行编译”说明安装正确
- 本页面还可以切换选择VS 2017/2015等其他编译器

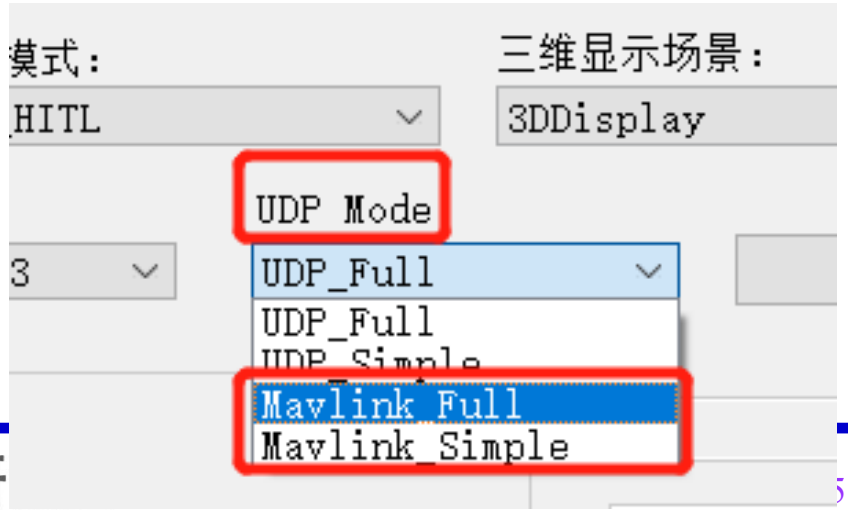


1.安装配置

1.3 集群例程文件夹介绍

- 本讲课程的视觉控制例程都存放在“RflySimAPIs\SimulinkSwarmAPI”路径下，里面的集群控制算法主要通过Simulink实现，同时也提供了Python接口。
- 如右图所示，文件夹内包含了“Full”和“Simple”两种后缀的bat启动脚本，这是对应了下图CopterSim中的“UDP_Mode”选项，针对不同规模的集群仿真的通信需求。
- “Full”模式(CopterSim默认)采用完整通信协议，数据详细，但飞机数量太多会导致宽带饱和而产生大延迟；大规模集群(单电脑飞机数≥8)仿真时推荐使用“Simple”模式，使用精简协议，传输的数据量少，延迟低。





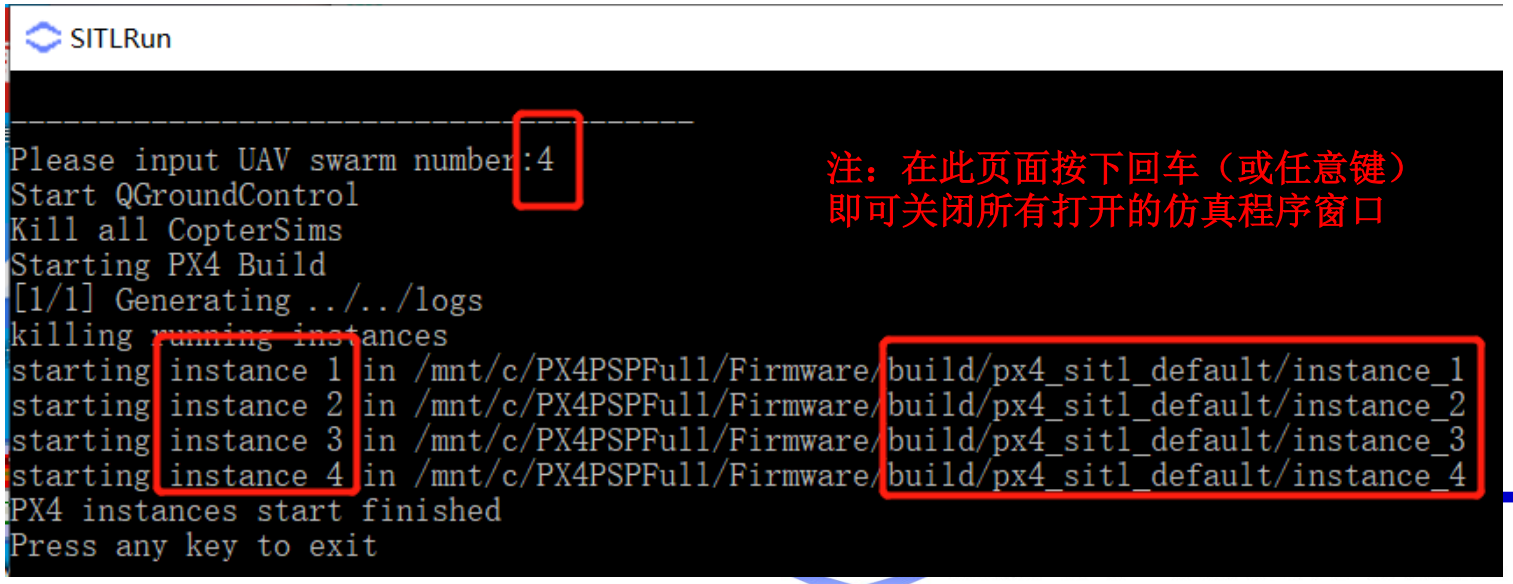
1.安装配置

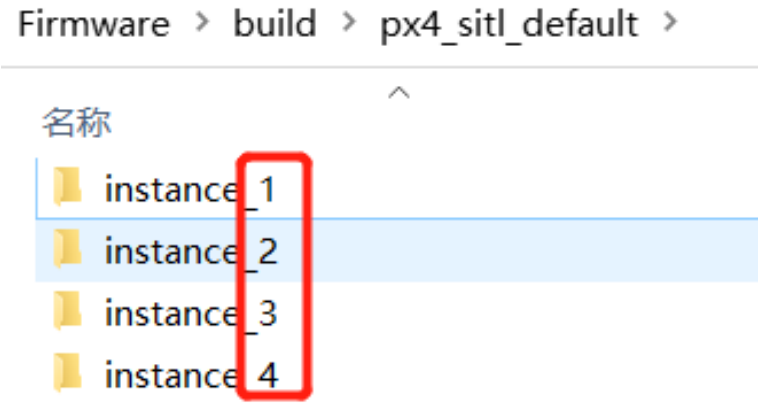




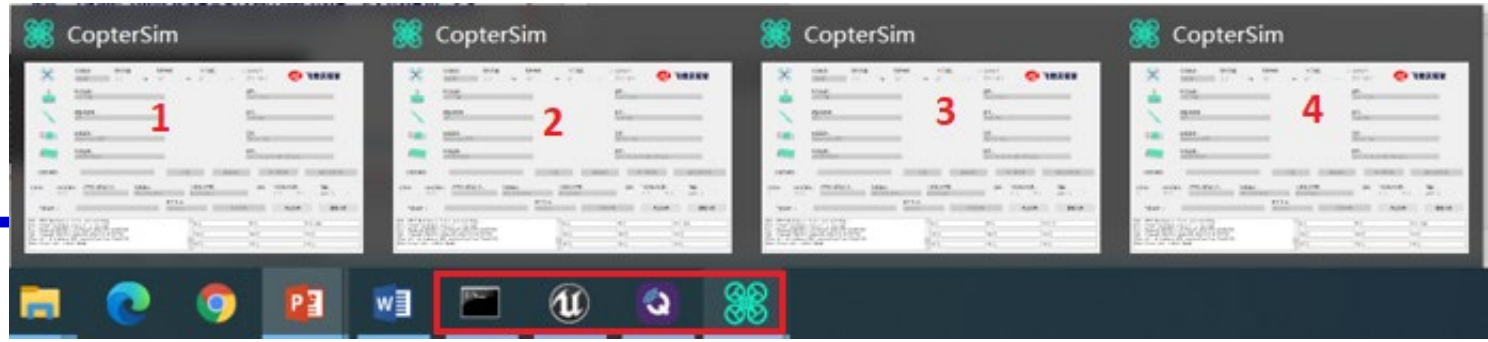
1.4 集群软件在环仿真测试

- 双击运行“RflySimAPIs\SITLRun.bat”（或桌面同名快捷方式），在弹出命令提示符中输入“4”，点击回车键，就能快速开启4个飞机的集群仿真系统
- 自动打开的软件包括4个CopterSim（每个飞机对应一个）、1个RflySim3D（会同时显示所有飞机）、1个QGC地面站（同时显示并控制所有飞机）和1个命令提示符窗口（调用Win10WSL编译器开启4个PX4 SITL控制器，每个控制器的log和参数可以访问Firmware\build\px4_sitl_default\instance_*）





1.安装配置



1.5 集群软件在环仿真结果

- 如右上图显示，Windows任务栏上可以看到所有打开的程序
- 如左下图所示，依次切换各个飞机（Vehicle 1~4），并依次点击“起飞”按钮，然后点“滑动来确认”，即可控制4个飞机依次起飞
- 起飞后再在RflySim3D中按下“S”键可显示各飞机ID，得到效果如右图



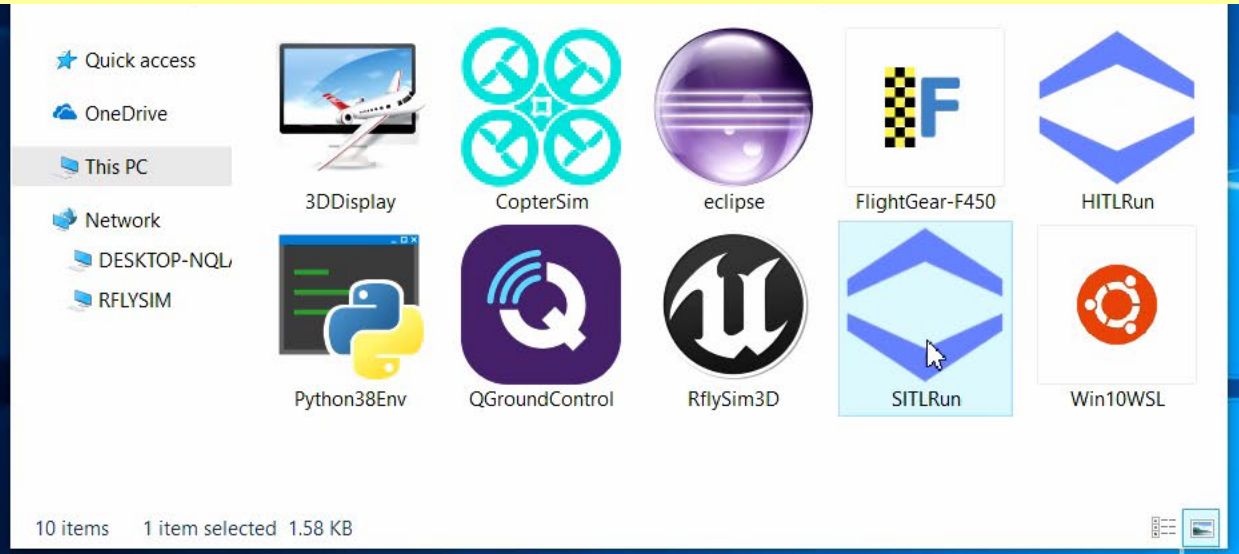


RflySim: 6. 如何快速启动多无人机集群的软件在环仿真

本视频观看地址：

优酷：https://v.youku.com/v_show/id_XNDcwNjA4MzA0NA==.html

YouTube：https://youtu.be/88dGpErxPJ8



点击SIL软件在环一键启动脚本并输入飞机数量4

1.安装配置

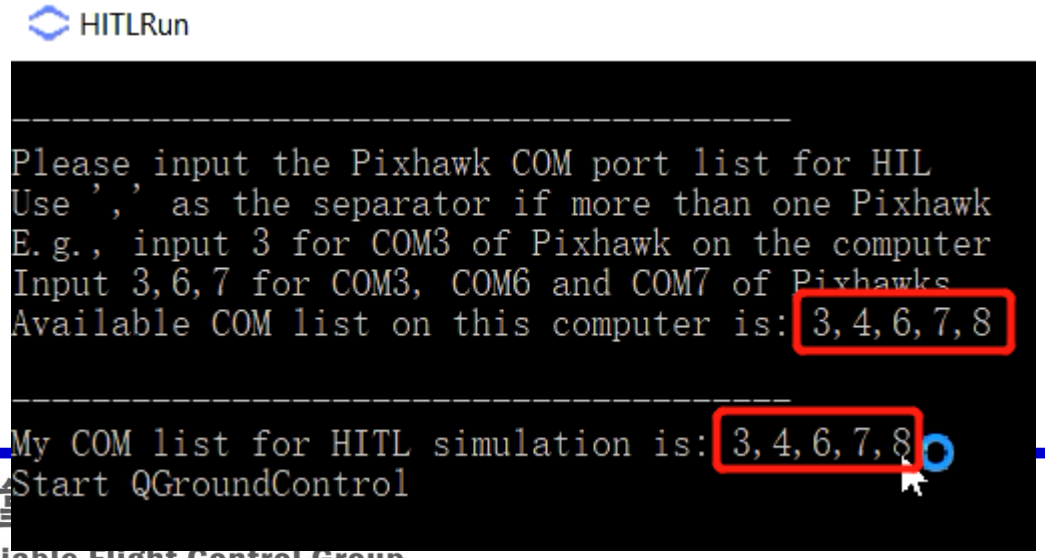




1.6 集群硬件在环仿真

- 如左下图所示，将所有Pixhawk飞控（请提前按高级版课程第1讲2.5~2.7小节步骤设置飞控到HITL模式）插入电脑
- 双击运行“RflySimAPIs\HITLRun.bat”（或桌面同名快捷方式），在右图所示弹出窗口中根据提示输入飞控串口号（英文逗号隔开），点击回车键，就能快速开启多个飞机的集群仿真系统
- 下面两个图对应了连接5个飞控进行硬件在环仿真的情形，运行结果和1.5节相同



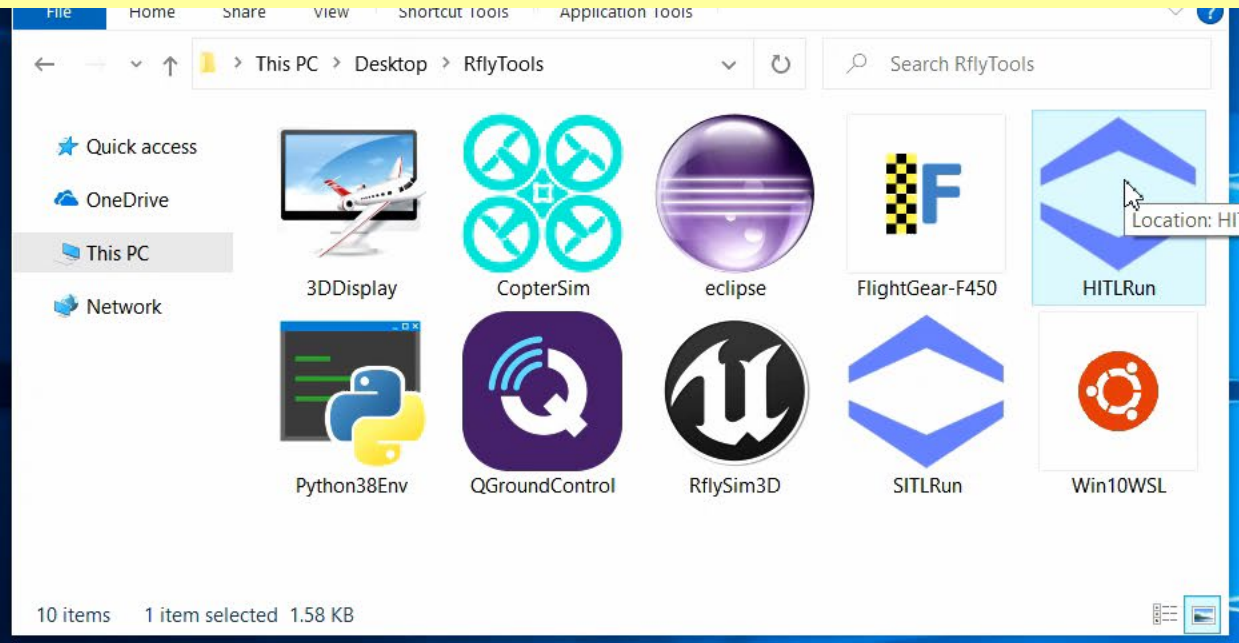


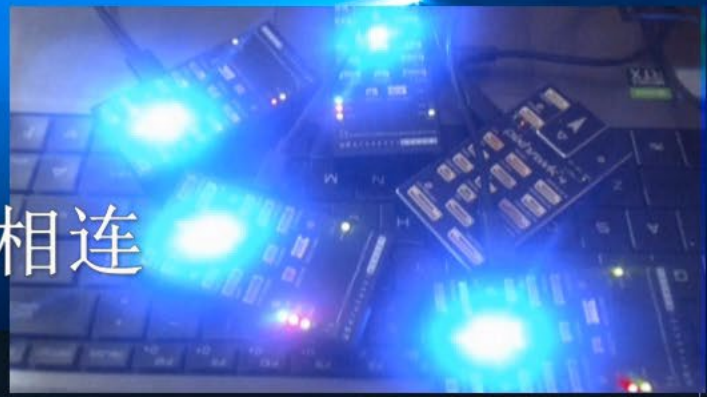
RflySim: 4. 如何快速启动多机的硬件在环编队仿真

本视频观看地址：

优酷：https://v.youku.com/v_show/id_XNDcwNjA3OTQ0MA==.html

YouTube：https://youtu.be/oZ_-yhEgebA





用USB线将所有Pixhawk与计算机相连

1.安装配置

1.7 RflySim3D快捷方式使用

F1：弹出帮助菜单提示；　　ESC：清除所有飞机

S：显示/隐藏飞机ID；　　M：切换地图；M+数字*：切换到地图*

B：视角在飞机间切换，序号从小到大切换；　B+数字**：切换到**号飞机

C：切换当前飞机样式；　C+数字**：切换到飞机样式**

V：飞机上的视角切换，0：跟随视角、1：前视摄像头、2：右视摄像头、...；V+数字**：切换到第**号视角

N：切换到飞机上帝视角，0：跟随飞机视角（不随飞机姿态改变视角角度）、1：固定地面视角且始终看向当前飞机、2：固定地面向北看视角、3：固定地面向南..

N+数字**：切换到第**个上帝视角

鼠标左键按下拖动：切换视角角度；鼠标右键按下拖动：切换视角所在纵向yz位置

鼠标滚轮：切换视角所在横向x位置

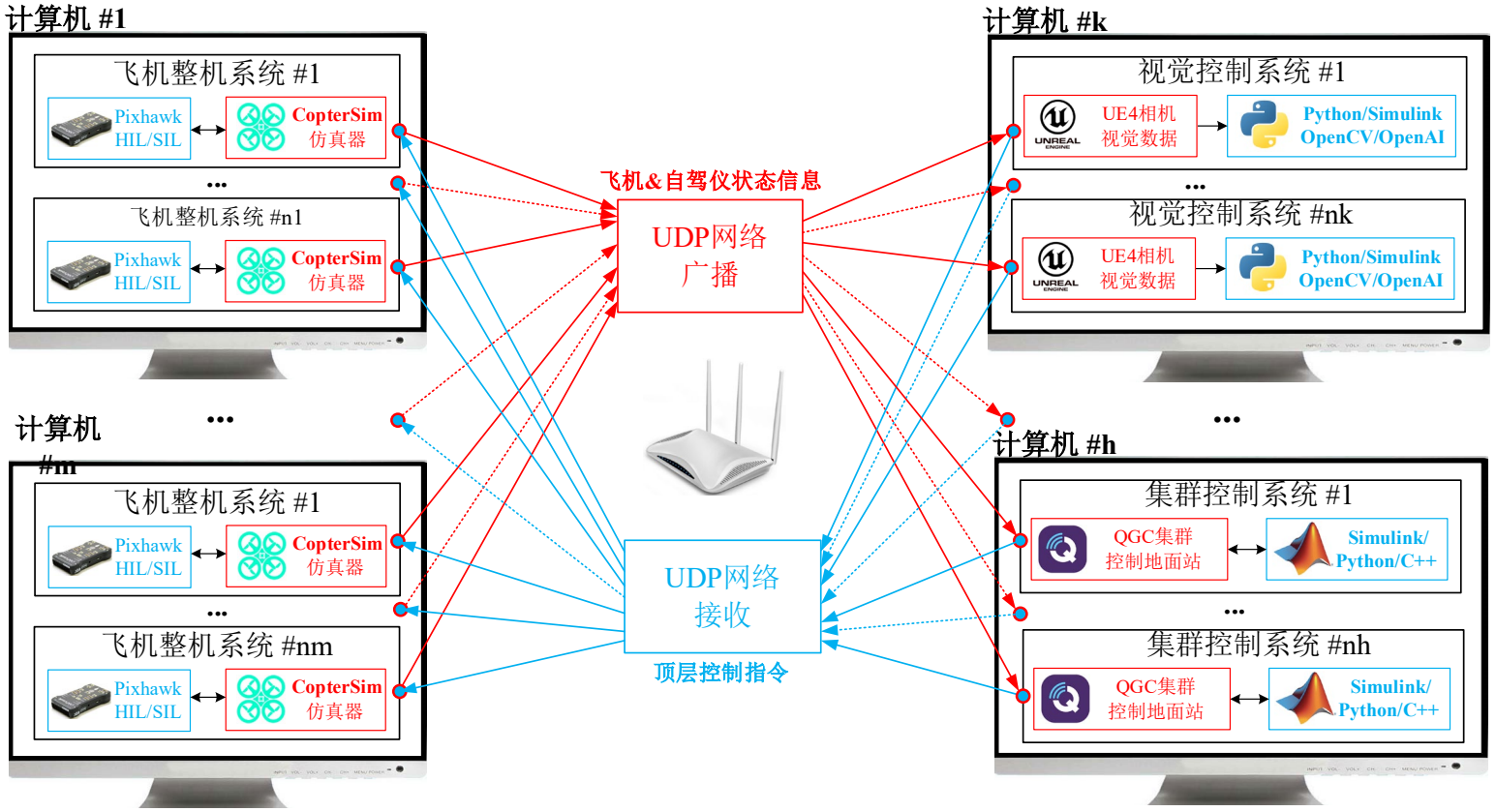
CTRL+鼠标滚轮：缩放所有飞机尺寸（多机时便于观察）；ALT+鼠标滚轮：缩放当前视角飞机尺寸

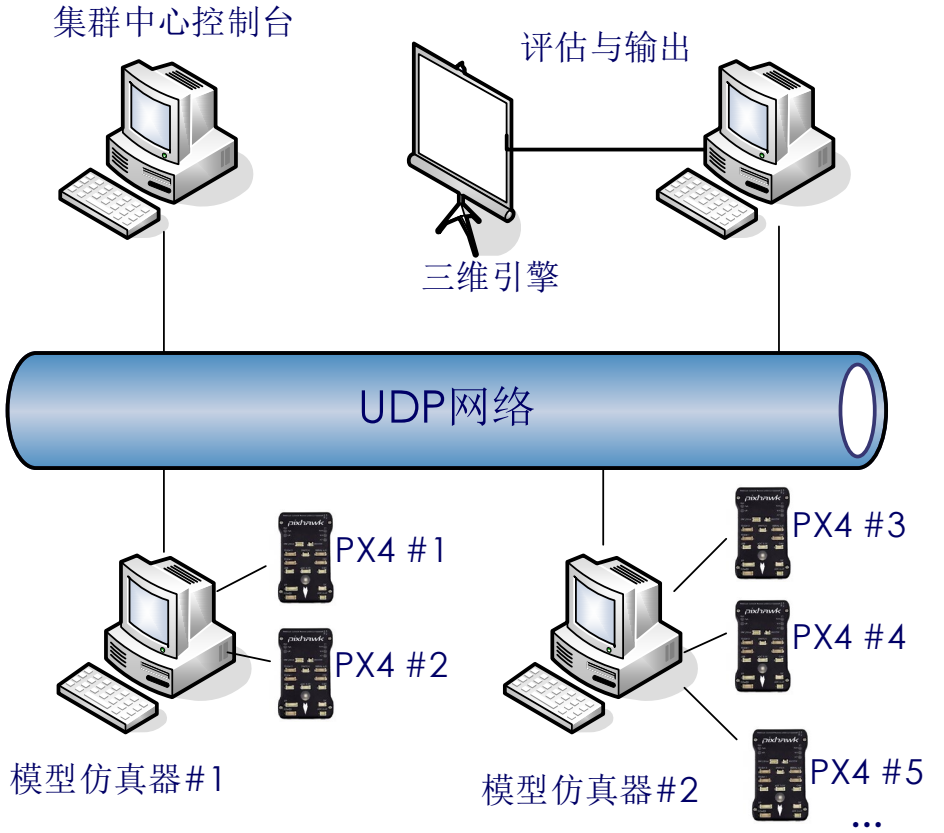
1.安装配置



1.8 RflySim集群系统分布式构架

- 每个APP都是独立进程，可在局域网电脑中多开
- 插入多个飞控可以进行多机的集群仿真
- 内部组网模拟实际硬件数传组网







1.安装配置

1.8 集群仿真难点与RflySim平台解决方式

- 问题一：启动N个飞机进行仿真太麻烦？
  解决方案：一键脚本启动所有程序并完成参数配置
- 问题二：多个飞机同时在同一场景显示？
  解决方案： UDP广播方式，RflySim3D根据接收飞机数据自动新建飞机，理论上可以扩展任意多个飞机，也支持新建障碍物、行人等辅助对象
- 问题三：飞机太多网络拥堵？
  解决方案：提供多种数据协议，大规模集群时可选用最精简数据模式，确保网络通畅
- 问题四：多机集群使用硬件在环的话成本高操作复杂？
  解决方案：支持PX4 SITL软件在环仿真，直接在电脑上新建并运行N个完整PX4控制器，且可访问每个控制器的log日志、飞行参数等
- 问题五：仿真平台性能如何？
  回答：主要是QGC和RflySim3D占用计算资源，模型和控制器占用资源较小。初步测试：配置较好的电脑软件在环仿真能稳定15架次以上，不运行QGC和RflySim3D数量增加，使用Pixhawk硬件在环仿真可再翻倍。

1.8 集群仿真难点与RflySim平台解决方式

- 问题六： Simulink随着控制飞机的增加速度变慢，无法实时控制？
  解决方案：支持Simulink自动代码生成exe可执行文件，电脑性能占用趋近于0
- 问题七：单电脑性能有限，只能仿真特定数量的飞机？
  解决方案：支持局域网内集群联机仿真，确保大规模集群顺利进行，支持局域网广播与指定IP方式，减小通信延迟
- 问题八：集群仿真如何与实际飞行尽量接近？
  解决方案1：Simulink集群接口接收的是Pixhawk通过Mavlink发送的内部估计状态而非仿真理想值，且发送控制指令也是Mavlink控制格式，因此集群控制器可直接用于真机实飞。
  解决方案2： Simulink控制器支持生成C/C++代码嵌入到原有集群系统中，也支持实飞时直接用Simulink接收局域网Pixhawk内发送的Mavlink消息进行集群控制。即：仿真完不需要修改可直接用于实飞。

大纲

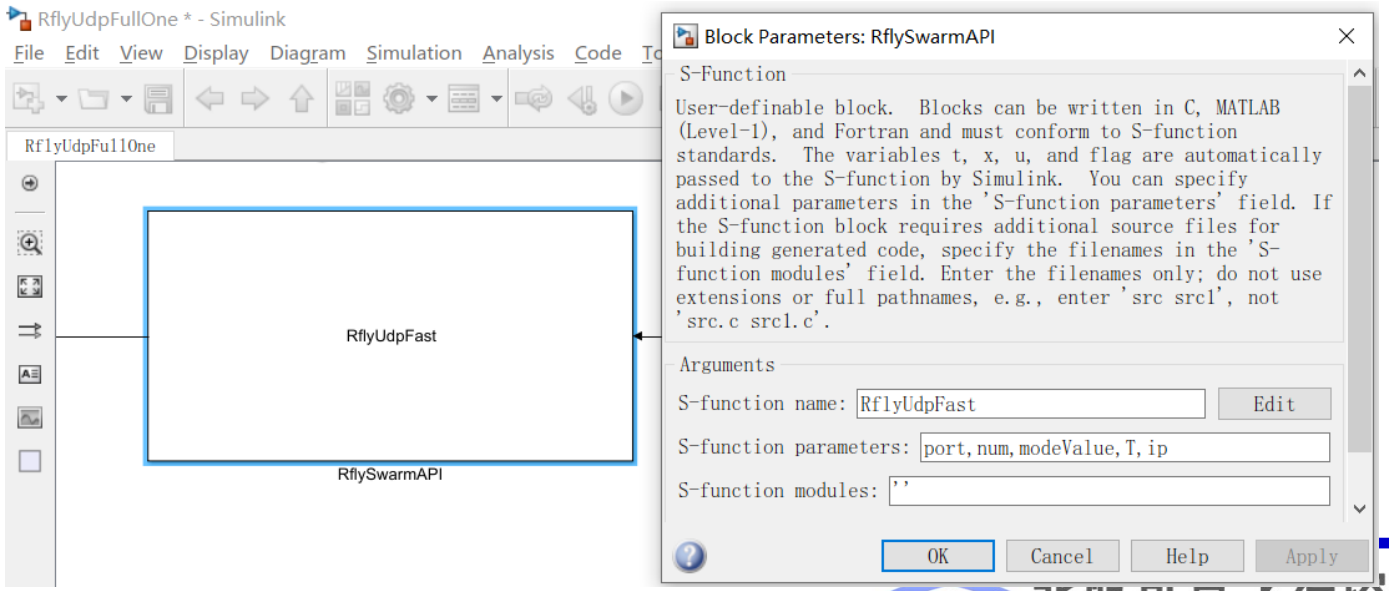
1. 安装配置
2. 接口介绍
3. 基础例子
4. 进阶例子
5. 小结

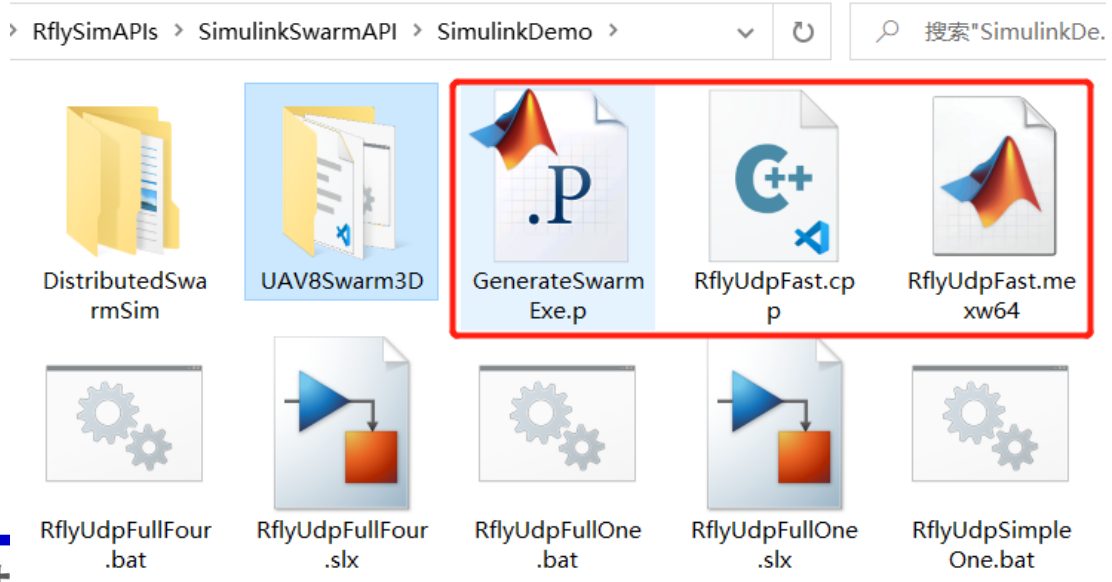
本讲的例程源码路径：
RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo

2.接口介绍

2.1 Simulink集群通信接口

- 平台提供Simulink和Python两种通信接口，底层都是通过MAVLink协议实现，因此仿真完的算法是可以很快地部署到实验平台上。这里先介绍Simulink的接口：
- 打开“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpFullOne.slx”文件，如左下图可看到“RflySwarmAPI”模块即为集群通信模块。
- 该模块有Simulink S函数通过C++混编实现，源文件见“RflyUdpFast.cpp”。

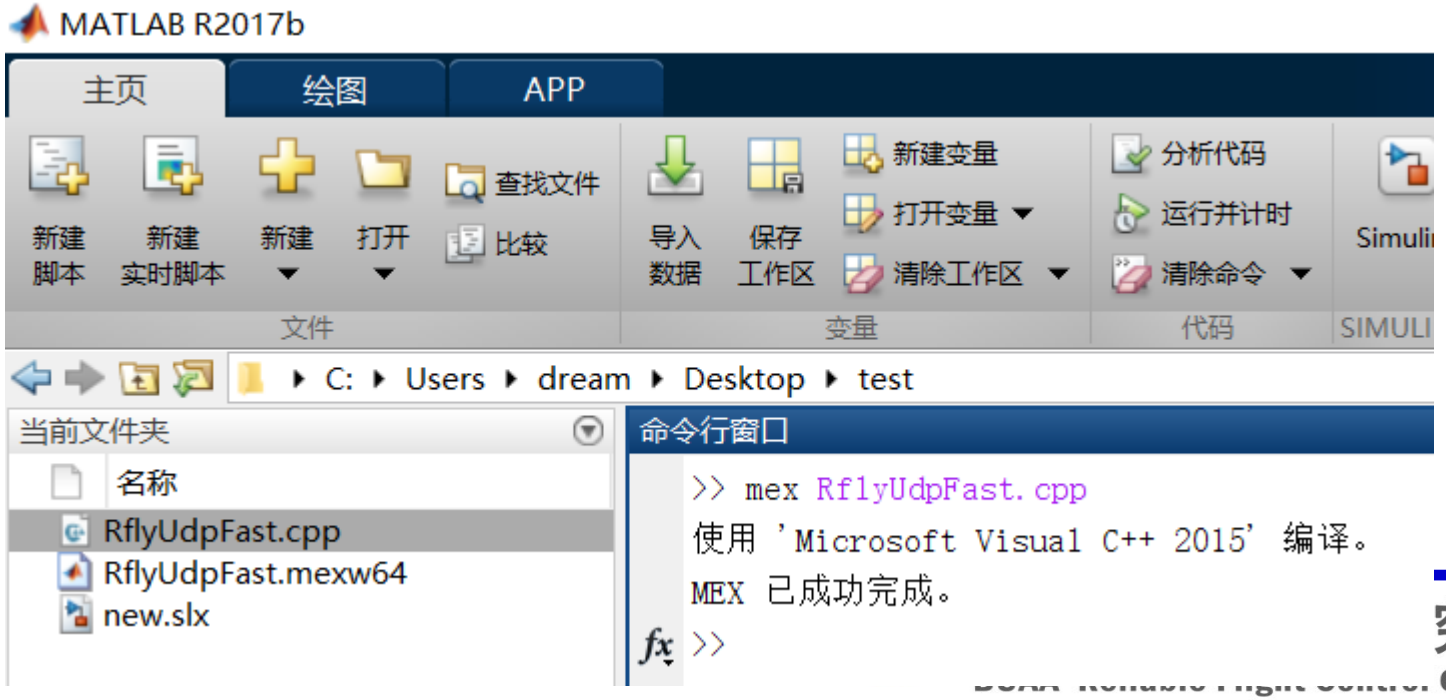




2.接口介绍

2.2 Simulink集群通信接口使用

- “RflySwarmAPI”模块需要和“RflyUdpFast.mexw64”放在同一个文件夹才能调用。因此新建slx项目时，拷贝“RflySwarmAPI”模块的同时还需要拷贝“RflyUdpFast.mexw64”文件到新建slx文件所在目录。
- “RflyUdpFast.mexw64”是源文件“RflyUdpFast.cpp”在MATLAB中编译出来的。如果修改了.cpp文件，还需要使用命令“mex RflyUdpFast.cpp”重新编译



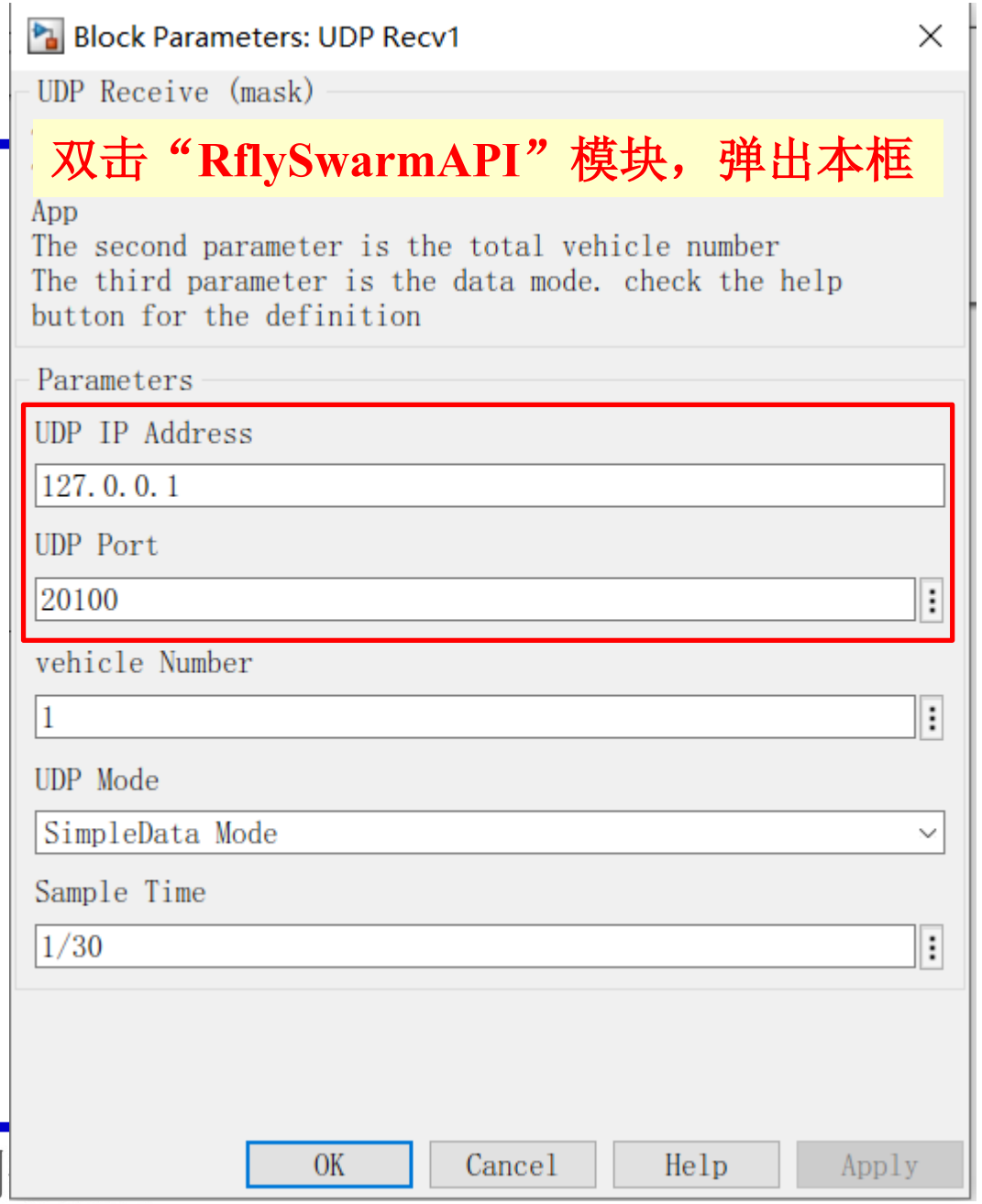
- 如左图，MATLAB输入“mex RflyUdpFast.cpp”（可自行开发ROS、串口、TCP等其他通信）可得到“RflyUdpFast.mexw64”文件
- 拷贝“RflyUdpFast.mexw64”到一个文件夹，新建一个new.slx (或其他名字)，将“RflyUdpFullOne.slx”中的“RflySwarmAPI”拷贝进去，在里面开发集群算法即可。

2.接口介绍

2.3 集群通信接口配置解析

a) 第一项“UDP IP Address”是目标电脑的IP地址，输入“127.0.0.1”则只能接受本机的CopterSim转发的Pixhawk自驾仪状态并进行控制；“255.255.255.255”则能接收并控制局域网内所以电脑中运行的CopterSim程序（其他电脑的CopterSim需要勾选“联机”按钮）；“192.168.1.12”之类的指定IP则只会向该IP地址的主机发送控制指令。一般而言，在小范围集群的时候“255.255.255.255”广播已经能够满足需求，当飞机数量继续增多，则需要启用指定IP来减小网络负载，提高通信速度和可靠性。

b) 第二项“UDP Port”是第一个飞机的初始端口号，默认起始端口是20100。每个CopterSim的收发消息需要各占用一个端口，因此如果本模块需要仿真飞机ID为10~15的飞机，那么这一项需要填20100+10*2=20120，后面一项“Vehicle number”飞机数量需要填5.



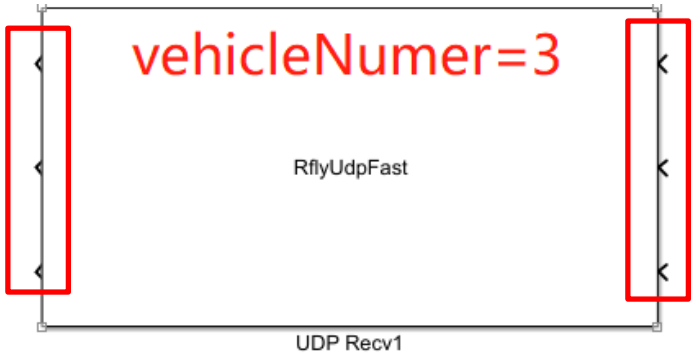
2.接口介绍

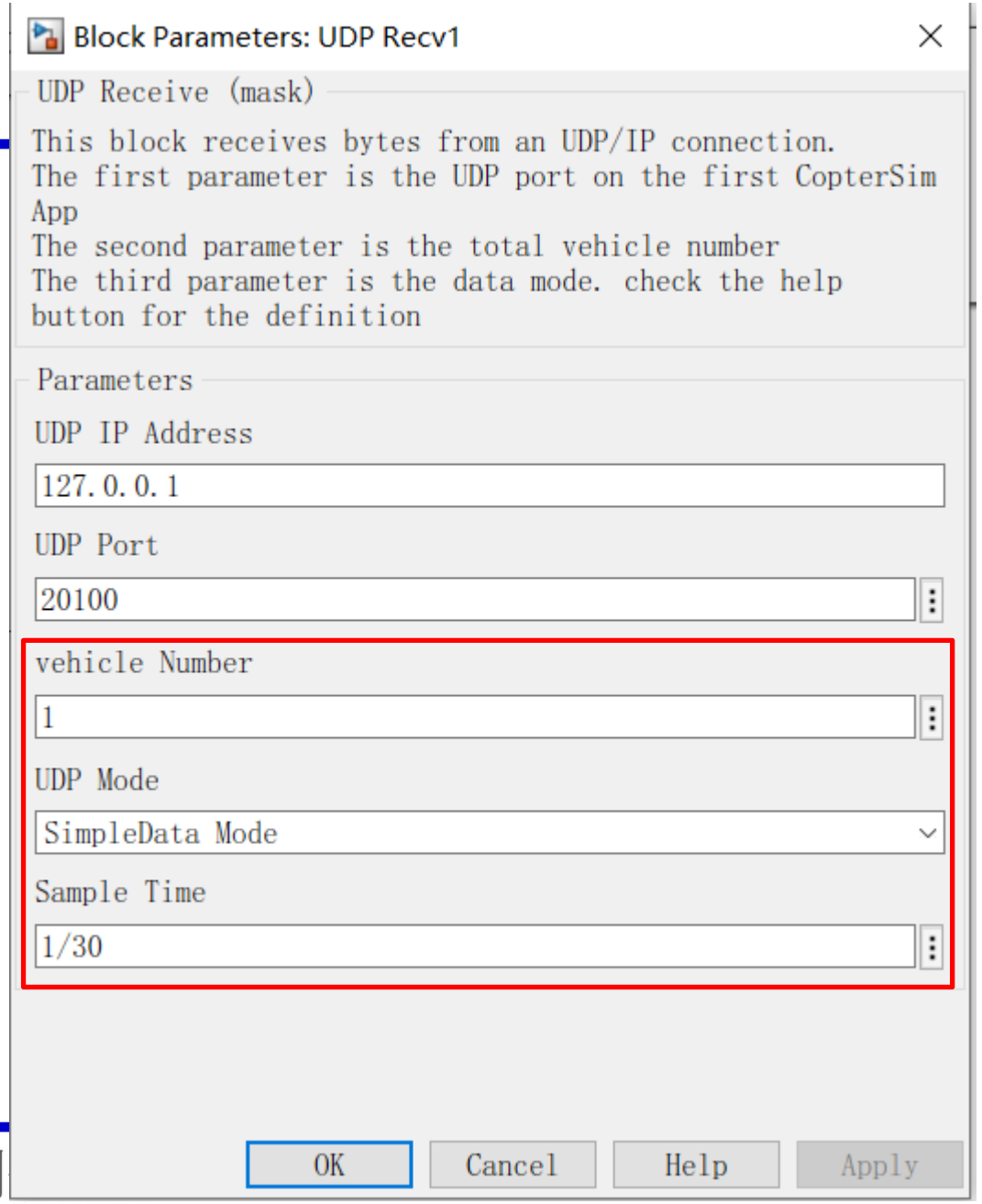
2.3 集群通信接口配置解析

c) 第三项“Vehicle number”飞机数量表示需要连接的CopterSim数量，该模块的输入输出端口数量由该选项控制，如果输入10，则模块会自动生成10对输入输出接口。

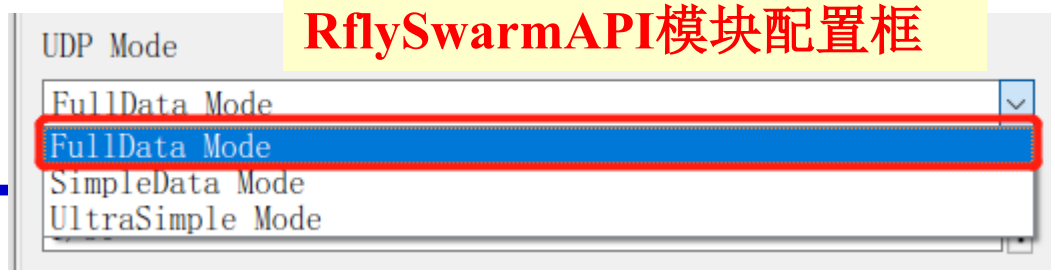
d) 第四项“UDP mode”是输入输出接口的数据模式协议，主要包含FullData完整模式（数据最全，但传输数据量较大）；SimpleData精简数据模式（较多飞机>8，避免数据过大网络阻塞）和UltraSimple超精简模式（单电脑飞机数>20），延迟更小。

e) 第五项是“Sample Time”采样时间，该时间应该与Simulink仿真时间对应。





2.接口介绍



2.4 通信接口的FullData模式 – 输入

- 模块输入为15维的double型向量，具体定义（实现MAVLink的Offboard消息）如下

① 第1维：time_boot_ms; %当前时间戳（填0即可，目前没有使用）

② 第2维：copterID; %飞机ID（填1即可，目前没有使用）

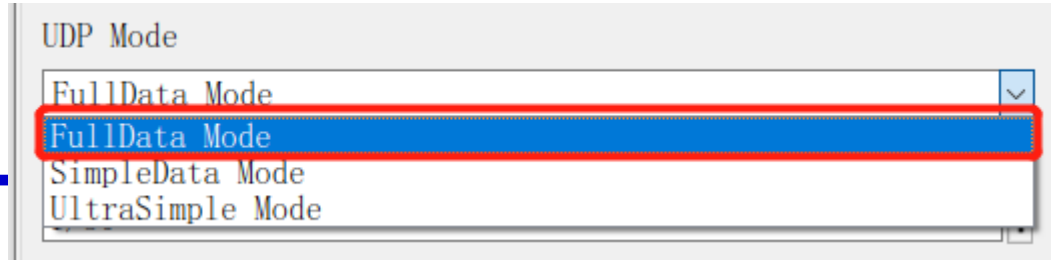
③ 第3维：type_mask; %输入控制模式（同Offboard定义）

④ 第4维：coordinate_frame; %坐标系模式（同Offboard定义）

⑤ 第5~15维：ctrls[11]; %分别对应了3维的期望位置pos,3维的期望速度vel，3维的期望加速度acc，1维的期望偏航角yaw，1维的期望偏航角速度yawRate。（同Offboard定义）

- 该模块与“RflySimAPIs\SimulinkControlAPI\OffboardAPI.slx”功能相同（见高级课程第3讲3.3节），实现了Offboard控制的所有功能，能实现位置、速度、加速度跟踪等。
- Offboard消息的详细定义可见
- https://mavlink.io/en/messages/common.html#SET_POSITION_TARGET_LOCAL_NED

2.接口介绍



2.5 通信接口的FullData模式 – 输出

- 模块输入为28维的double型向量（全部转发自Pixhawk内部滤波值），具体定义如下

① 第1~3维：gpsHome[3]; %Home点（上电之后不会变）的经纬高坐标，经纬度需要除以1e7才能得到度为单位的经纬度，高需要除以1e3才能得到m为单位的高（向上为正）第2维：copterID; %飞机ID（填1即可，没有用到）

② 第4~6维：AngEular[3]; %Pixhawk估计得到的姿态欧拉角，单位弧度

③ 第7~9维：localPos[3]; %Pixhawk估计得到的以gpsHome为原点的相对北东地位置向量，单位m，z轴向下为正

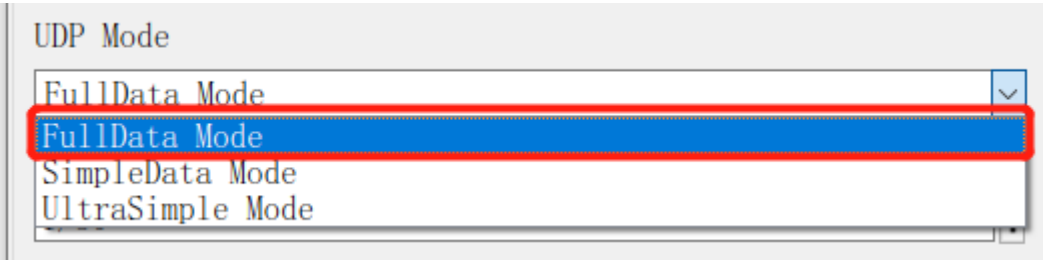
④ 第10~12维：localVel[3]; %北东地的运动速度向量，单位m/s

⑤ 第13~15维：GpsPos[3]; %实时的GPS位置，单位和gpsHome相同，但是会实时变化

⑥ 第16~18维：GpsVel[3]; %GPS速度，需要除以100得到m/s为单位的速度

⑦ 第19维：time_boot_ms; %上电时间

2.接口介绍



2.5 通信接口的FullData模式 – 输出

- 模块输入为28维的double型向量，具体定义如下

⑧ 第20维： copterID; %飞机ID

⑨ 第21维： relative_alt; % GPS相对高度，需要除以1000得到m为单位的高度，向上为正

⑩ 第22维： hdg; % GPS航向角，需要除以1000得到0~360度范围的角度

⑪ 第23维： satellites_visible; %可见卫星数量

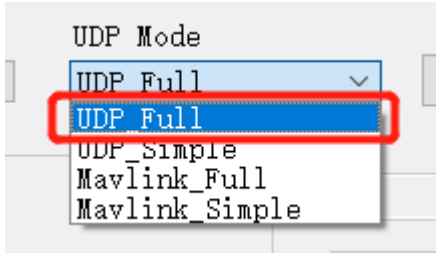
⑫ 第24维： fix_type; %定位精度

⑬ 第25维： resrveInit; % int类型的保留位

⑭ 第26维： pos_horiz_accuracy; %水平定位精度，单位m

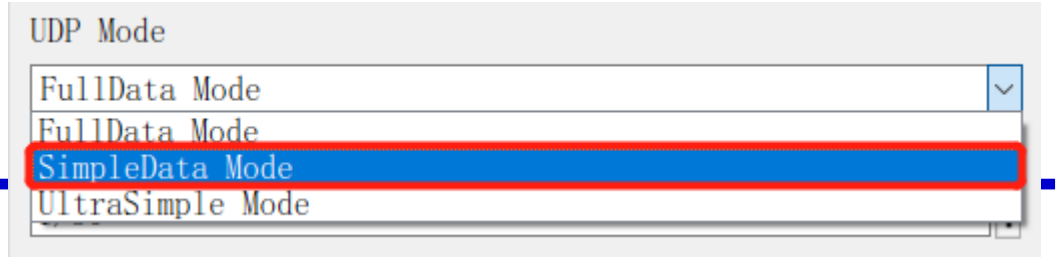
⑮ 第27维： pos_vert_accuracy; %竖直定位精度，单位m

⑯ 第28维： resrveFloat; % float型保留位，未被启用



- 使用本模式，CopterSim上的“UDP Mode”选项要选择“UDP_Full”

2.接口介绍



2.6 通信接口的SimpleData模式 – 输入

- 输入为5维的double型向量，具体定义（实现MAVLink的Offboard消息）如下

① 第1维：ctrlMode; %第一位为标志位，0: 表示地球速度控制模式Earth Vel; 1: 机体速度控制模式Body Vel; 2: 地球位置控制模式Earth Pos; 3:机体位置控制模式Body Pos第2维：copterID;

② 第2~5维：如果ctrlMode =0，则这四维对应了地球坐标系（以解锁时的位置）下的x,y,z,yaw位置+偏航信号；如果ctrlMode =1，则这四维对应了机体坐标系（以解锁时机体姿态和位置）下的x,y,z,yaw位置+偏航信号；如果ctrlMode =2，则这四维对应了地球坐标系（以解锁时的位置为原点）下的vx,vy,vz,yawRate位置+偏航信号；如果ctrlMode =3，则这四维对应了机体坐标系（以解锁时机体姿态和位置）下的vx,vy,vz,yawRate位置+偏航信号。

③ 一般ctrlMode=0比较多，直接给定全局速度做轨迹控制。如果做位置和速度的切换，需要实时修改ctrlMode的值，并调整第2~5维信号的定义。

2.接口介绍



2.7 通信接口的SimpleData模式 – 输出

- 输出为12维double型的向量，顺序定义如下

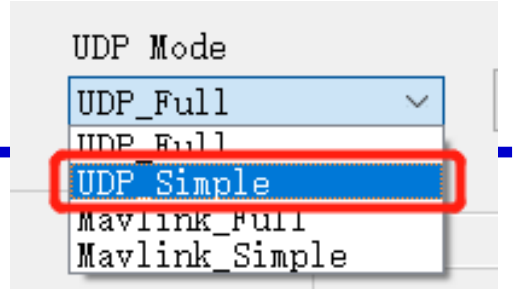
① 第1~3维： gpsHome[3]; % Home点（上电之后不会变）的经纬高坐标，经纬度需要除以1e7才能得到度为单位的经纬度，高需要除以1e3才能得到m为单位的高（向上为正）

② 第4~6维： AngEular[3]; % Pixhawk估计得到的姿态欧拉角，单位弧度

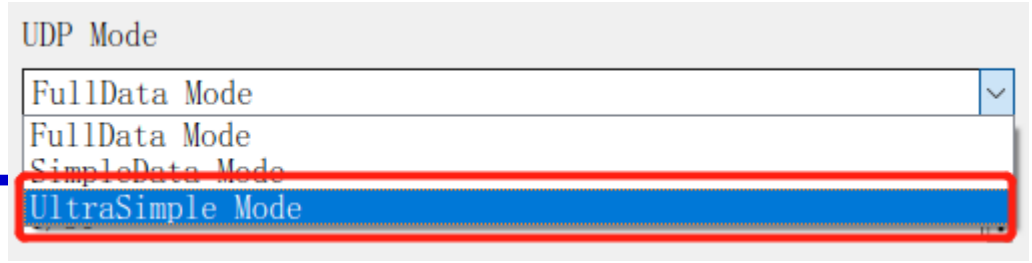
③ 第7~9维： localPos[3]; % Pixhawk估计得到的，以gpsHome为原点的相对北东地位置向量，单位m，z轴向下为正

④ 第10~12维： localVel[3]; % 北东地的运动速度向量，单位m/s

- 注： gpsHome为经纬高坐标，如果要换算成CopterSim中的地图坐标需要减去其初始点经纬高（116.2593683°，40.1540302°，0m），再换算成单位m
- 注：SimpleData模式不仅是输入输出精简了，UDP传输的结构体也精简，因此没有FullData模式中的其他信息。使用本模式，CopterSim上的“UDP Mode”选项要选择“UDP_Simple”



2.接口介绍



2.8 通信接口的UltraSimple模式

- 输入为5维的double型向量，具体定义与2.5节中的SimpleData模式完全相同
- 输出为12维double型的向量，顺序定义如下

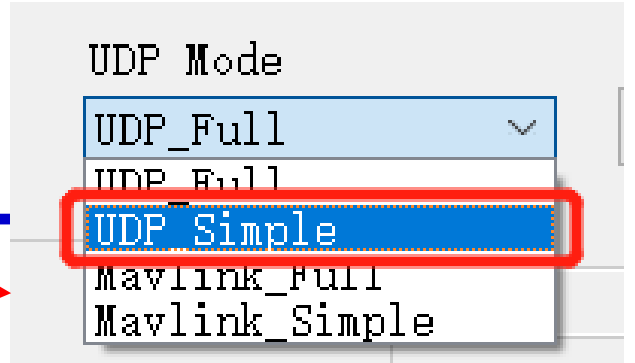
① 第1~3维： GlobalPos [3]; % CopterSim全局位置，单位m

② 第4~6维： AngEular[3]; % Pixhawk估计得到的姿态欧拉角，单位弧度

③ 第7~9维： localPos[3]; % Pixhawk估计得到的，以gpsHome为原点的相对北东地位置向量，单位m，z轴向下为正

④ 第10~12维： localVel[3]; % 北东地的运动速度向量，单位m/s

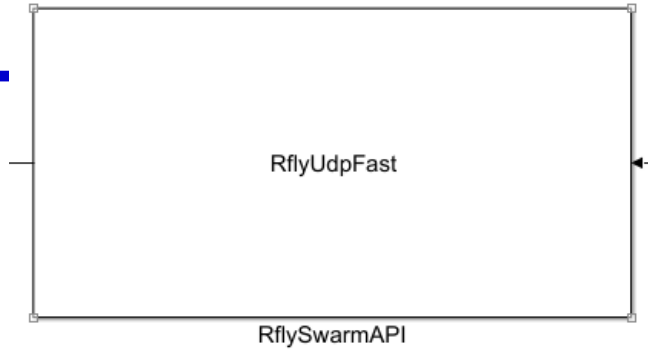
- 注：UltraSimple模式和SimpleData模式的唯一区别在于输出的第1~3维从原来的gpsHome变为了GlobalPos，其中GlobalPos是gpsHome经过经纬高的坐标转换之后，计算得到的CopterSim中的全局x,y,z（北东地坐标系）坐标
- 注：SimpleData和UltraSimple模式的内网传输数据相同，CopterSim上的“UDP Mode”选项要选择“UDP_Simple”

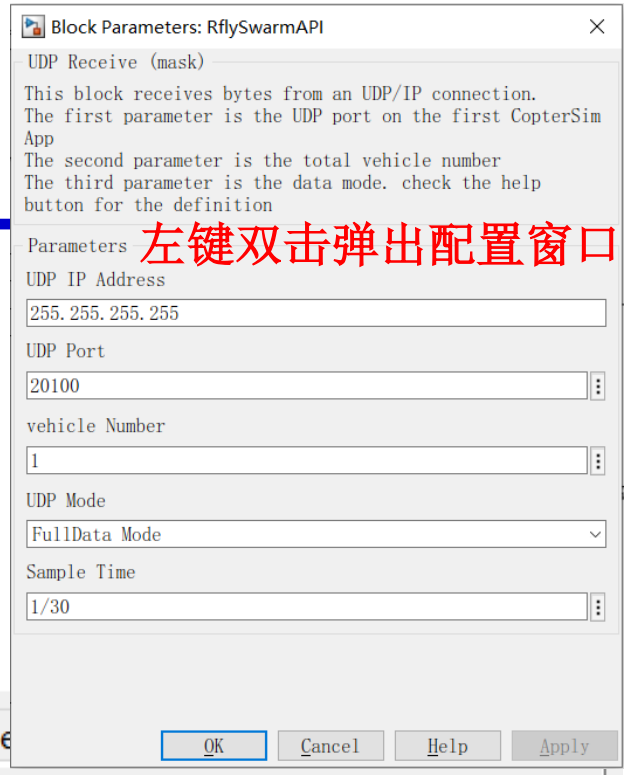


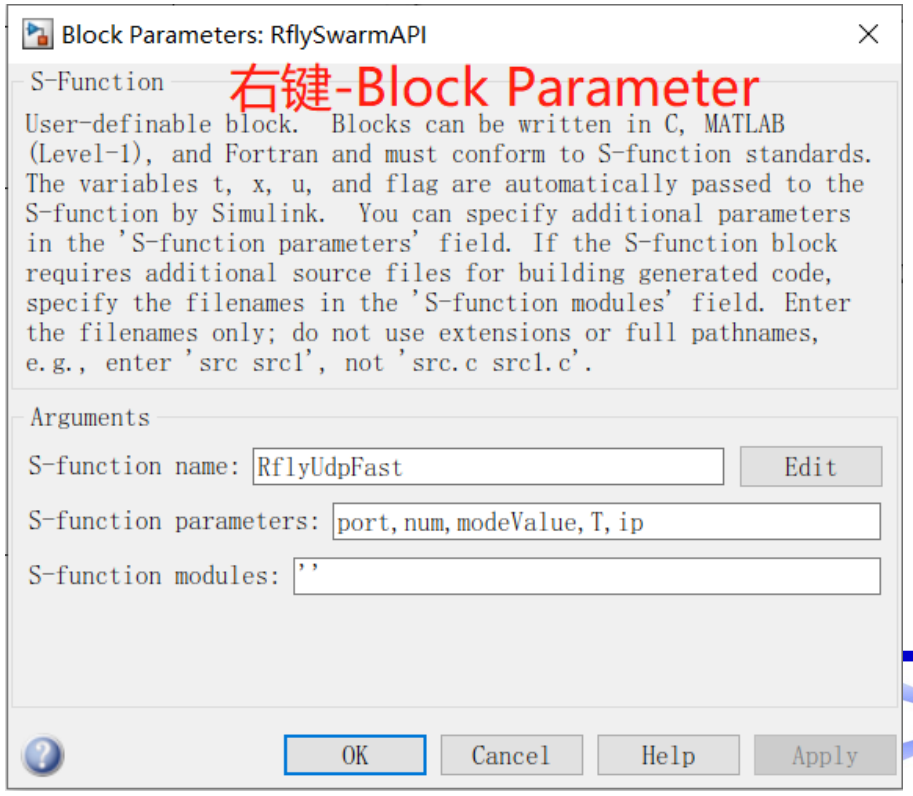
2.接口介绍

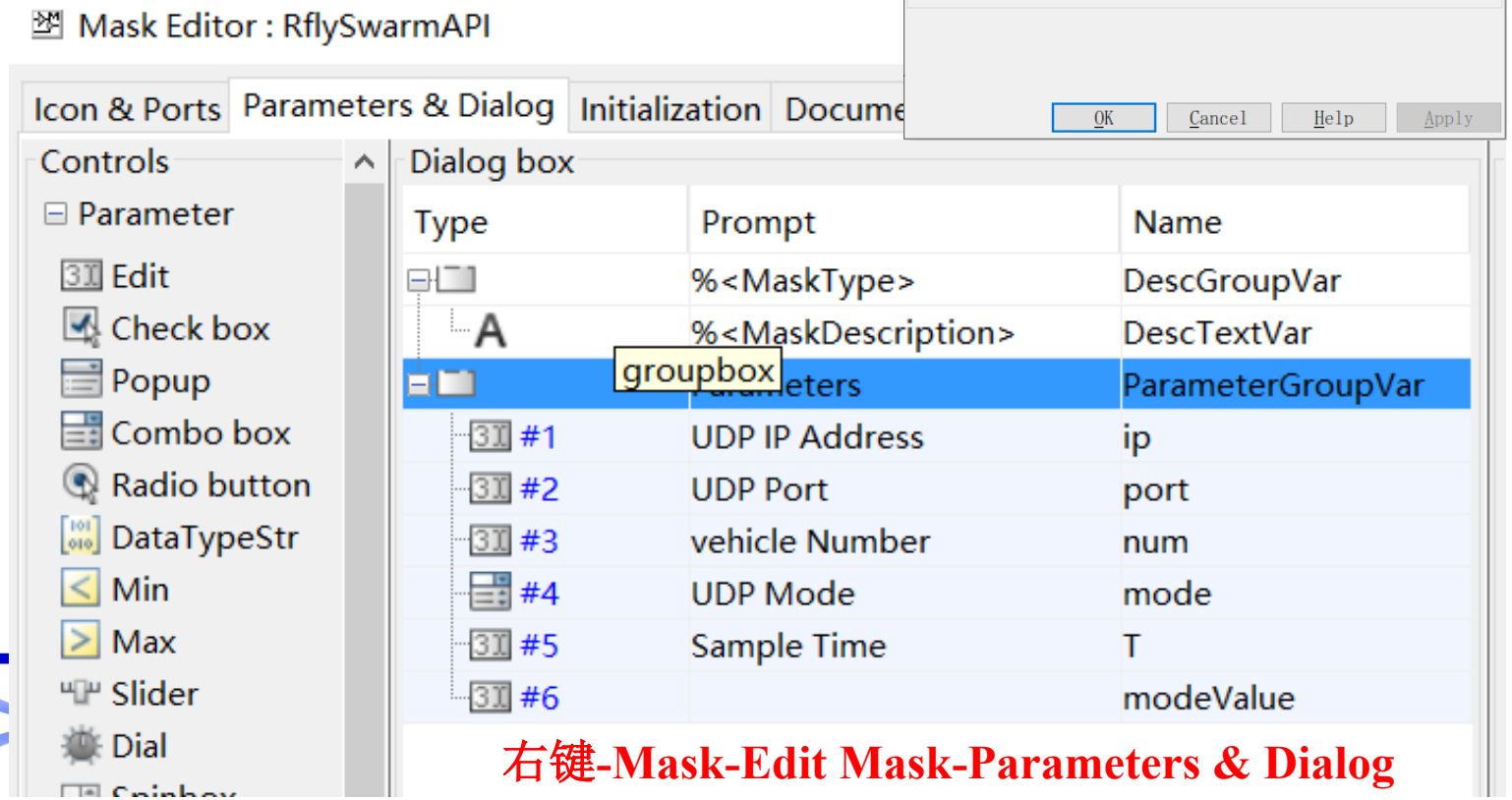
2.9 RflySwarmAPI模块实现原理解析

- 本模块调用的S函数由左下图模块参数页面设置
- 本模块右图的配置菜单由右下图的Mask编辑器设定。

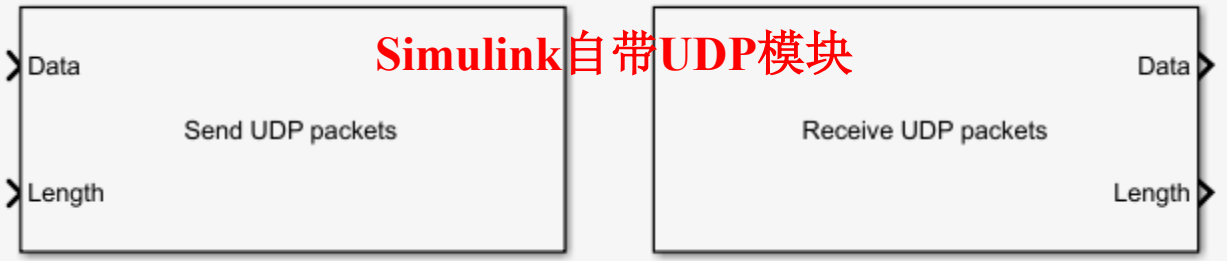








2.接口介绍



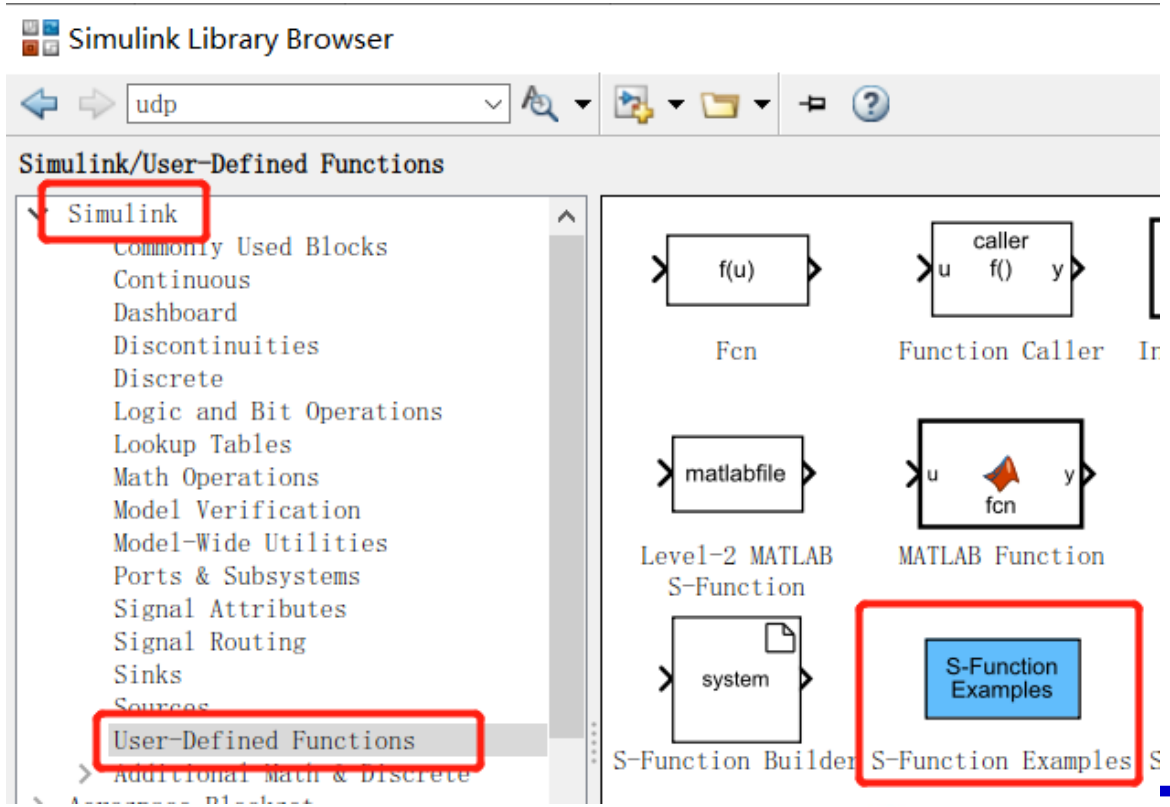
2.10 C++ S函数通信模块的优点

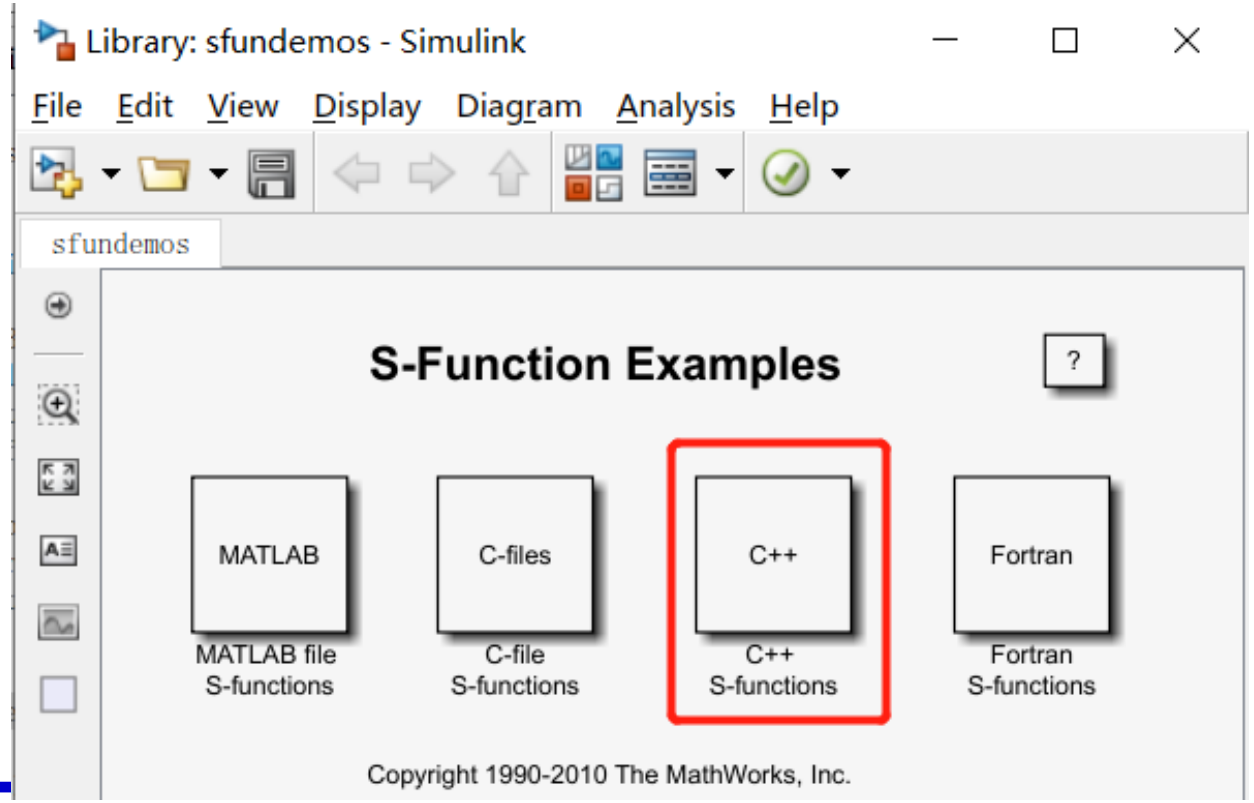
- 主要内容包括两个方面：UDP网络通信编程+Simulink S函数编写
- 本模块与Simulink自带的UDP模块的优点
- 效率高：C语言直接实现，比m语言高效
- 运算小：Simulink自带UDP模块输入输出都是高维（几百维）的unit8向量，在飞机增多时，整个项目维度急剧扩张；而S函数方式直接输出double型的位置、速度等数据，维度低（几维），运算量小
- 延迟低：Simulink自带的UDP模块为了防止数据丢失，会将收到数据全都保存在缓存中，依次调用，这样当外部程序发送频率大于Simulink的运行频率是，就会产生大的延迟；而编写S函数的方法则能避免本问题
- 更可靠：Simulink自带的UDP模块每个仿真步长读取一个数据，如果外部程序发送频率小于Simulink运行频率，则Simulink没有读到数据，会输出0，导致运算出错；而自己编写S函数则可以避免本问题。
- 扩展性强：S函数可以轻易扩展为串口、Tcp、共享内存、MAVLink等其他通信协议

2.接口介绍

2.11 S函数学习例子

- Simulink库浏览器-Simulink-User Defined Functions – S-Function Examples – C++ S-functions，从其中可以查看S函数编写方法，轻松实现Simulink控制其他系统

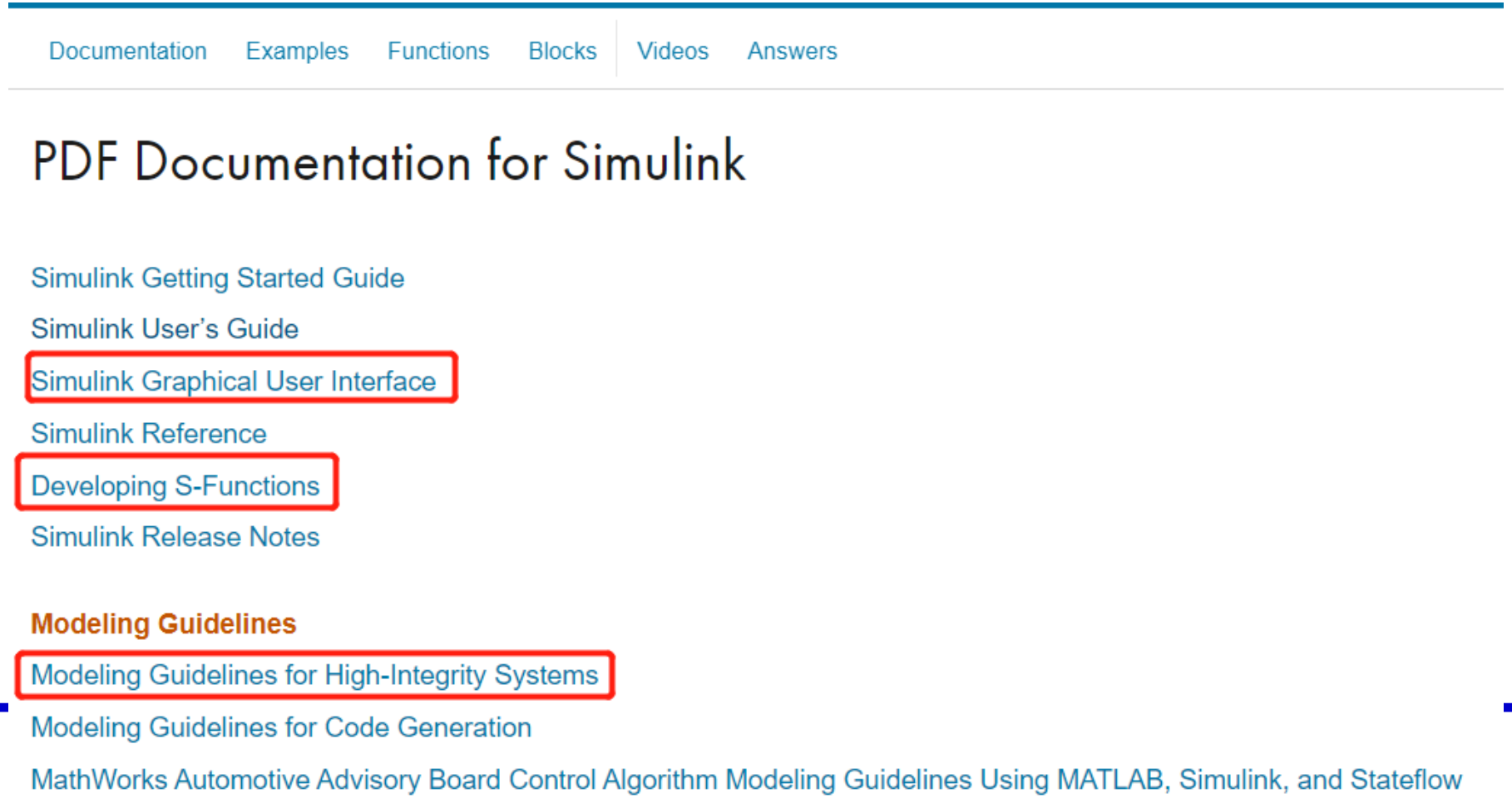




2.接口介绍

2.12 S函数官方PDF文档

- **https://www.mathworks.com/help/pdf_doc/simulink/index.html**



2.接口介绍

2.13 RflyUdpFast.cpp代码解析

RflyUdpFast.cpp

C: > PX4PSPFull > RflySimAPIs > SimulinkSwarmAPI > SimulinkDemo > RflyUdpFast.cpp > ...

```
1   /*  RflySim Simulink UDP Interface ***********************
2   /*  Xunhua Dai, 2020/05/03, Beihang University ***********
3
4   #define S_FUNCTION_NAME RflyUdpFast
5   #define S_FUNCTION_LEVEL 2
6   #define nonblockingsocket(s) {unsigned long ctl = 1;ioctlsocket(
7   #define _WINSOCK_DEPRECATED_NO_WARNINGS 1
8
9   #define MAXLEN 65536
10
11  #include "simstruc.h"
12  #include <math.h>
13  #include <stdio.h>
14  #include <string.h>
15  #include <winsock2.h>
16  #pragma comment(lib,"ws2_32.lib")
17
18  #define PAYLOAD_LEN_SHORT_SHORT 112
19
20  typedef signed char         int8_t;
21  typedef short               int16_t;
22  typedef int                 int32_t;
23  typedef long long           int64_t;
24  typedef unsigned char       uint8_t;
25  typedef unsigned short      uint16_t;
26  typedef unsigned int        uint32_t;
27  typedef unsigned long long uint64_t;
```

定义区：S函数的名字、类型，用户自定义其他常值变量

导入区：导入通信接口的库头文件

其他定义区，映射一些数据类型

```
30  struct outHILStateData{
31      uint32_t time_boot_ms; //消息时间戳ms
32      uint32_t copterID;     //飞机ID
33      int32_t GpsPos[3];     //滤波后的GPS经纬度，其中经纬度(度*1e7)，高度向上为正(单位m*1e3->mm)
34      int32_t GpsVel[3];     //滤波后的GPS速度，北东地，(单位m/s*1e2->cm/s)
35      int32_t gpsHome[3];     //GPS原始数据，其中经纬度(度*1e7)，高度向上为正(单位m*1e3->mm)
36      int32_t relative_alt;  //滤波后的GPS相对高度，北东地，(单位m*1e3->mm)
37      int32_t hdg;           //滤波后的GPS航向角，北东地，(单位m*1e3->mm)
38      int32_t satellites_visible; //GPS原始数据，卫星数量
39      int32_t fix_type;      //GPS原始数据，定位精度等级，3表示fixed
40      int32_t resrveInit;        //保留位Int
41      float AngEular[3];     //滤波后的飞机欧拉角，单位rad
42      float localPos[3];     //滤波后的本地位置，单位m
43      float localVel[3];     //滤波后的本地位置，单位m/s
44      float pos_horiz_accuracy;   //滤波器状态，水平定位误差，单位m
45      float pos_vert_accuracy; //滤波器状态，竖直定位误差，单位m
46      float resrveFloat;      //保留位Float
```

CopterSim中转发Pixhawk内部状态通信结构体定义

```
80  typedef struct _netDataShortShort {
81      int tg;
82      int        len;
83      char       payload[PAYLOAD_LEN_SHORT_SHORT];
84      _netDataShortShort() {
85          memset(payload, 0, sizeof(payload));
86      }
87  }netDataShortShort;
88
89  struct outHILStateShort{
90      int checksum;
91      int32_t gpsHome[3];     //GPS原始数据，其中经纬度(度*1e7)，高度向上为正(单位m*1e3->mm)
92      float AngEular[3];     //滤波后的飞机欧拉角，单位rad
93      float localPos[3];     //滤波后的本地位置，单位m
94      float localVel[3];     //滤波后的本地位置，单位m/s
95      outHILStateShort() {
```

其他通信结构体定义

2.接口介绍

2.13 RflyUdpFast.cpp代码解析

```
/* mdlCheckParameters, check parameters, this routine is called later fro
#define MDL_CHECK_PARAMETERS
static void mdlCheckParameters(SimStruct *S)
{
    /* Basic check : All parameters must be real positive vectors
    double *dPort, *dUdpMode, *numVehicle;
    int iPort, iUdpMode, iNumVehicle;
```

模块参数检验函数

```
240
241 /* mdlInitializeSizes - initialize the sizes array **********************
242 static void mdlInitializeSizes(SimStruct *S)
243 {
244     //int iBufSize;
245
246     ssSetNumSFcnParams(S,5);                          /* number of expec
247
248     /* Check the number of parameters and then calls mdlCheckParameters
249     if (ssGetNumSFcnParams(S) == ssGetSFcnParamsCount(S))
250     { mdlCheckParameters(S); if (ssGetErrorStatus(S) != NULL) return; }
251
```

初始化函数

```
408 void StartUDPServer(SimStruct *S, SInfo *info, int index)
409 {
410     double *dPort;
411     int iPort;
412
413     /* get the port number */
414     dPort=(double *)mxGetPr(ssGetSFcnParam(S,0));
415     iPort = (int)floor((*dPort)+0.5) +1 + (index-1)*2;
416
```

开始仿真函数

```
472 /* mdlOutputs - compute the outputs ************************************
473 static void mdlOutputs(SimStruct *S, int_T tid)
474 {
475     //int iBufSize = (int)floor((*mxGetPr(ssGetSFcnParam(S,2)))+0.5);
476     //UNUSED(tid);
477     UNUSED_ARG(tid);
478     int iBufSize =240;
479     double *numVehicle=(double *)mxGetPr(ssGetSFcnParam(S,1));
480     int iNumVehicle = (int)floor((*numVehicle)+0.5);
481
482     double *dUdpMode = (double *)mxGetPr(ssGetSFcnParam(S,2));
483     int iUdpMode = (int)floor((*dUdpMode)+0.5);
```

更新输出函数

```
594 #define MDL_UPDATE
595 /* Function: mdlUpdate ========================================
596  * Abstract:
597  *      xdot = Ax + Bu
598  */
599 static void mdlUpdate(SimStruct *S, int_T tid)
600 {
601     UNUSED_ARG(tid);
602     double *numVehicle=(double *)mxGetPr(ssGetSFcnParam(S,1));
603     int iNumVehicle = (int)floor((*numVehicle)+0.5);
604
605     double *dUdpMode = (double *)mxGetPr(ssGetSFcnParam(S,2));
606     int iUdpMode = (int)floor((*dUdpMode)+0.5);
```

更新状态函数

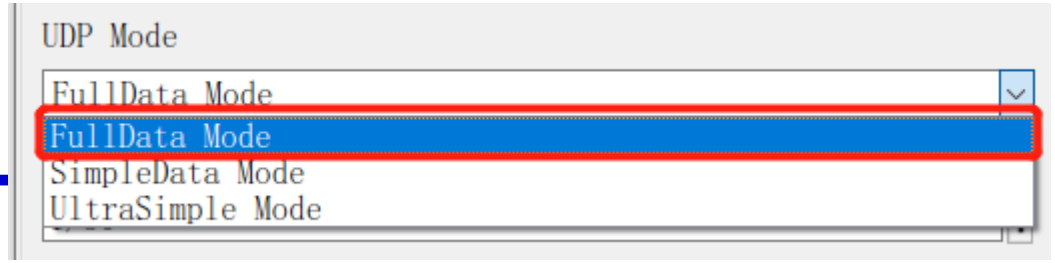


大纲

1. 安装配置
2. 接口介绍
3. 基础例子
4. 进阶例子
5. 小结

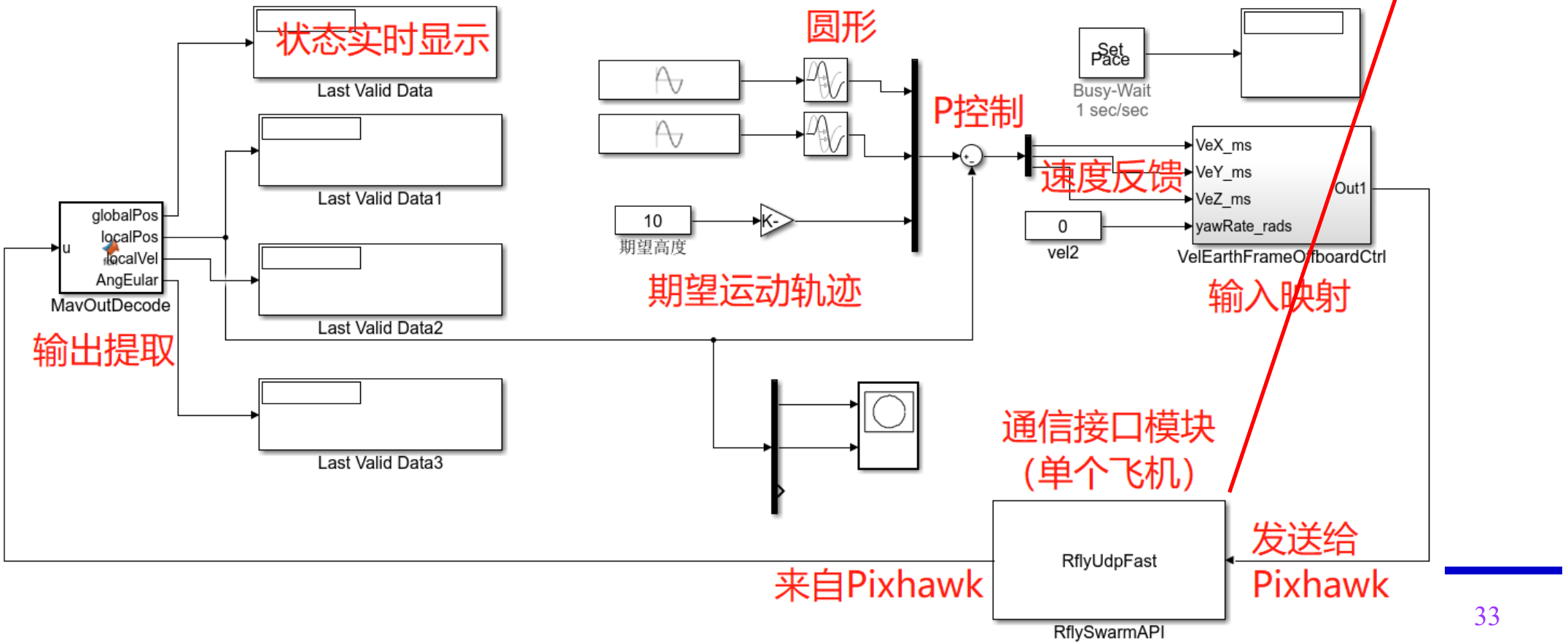
本讲的例程源码路径：
RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo

3.基础例子



3.1 通信接口FullData模式例子

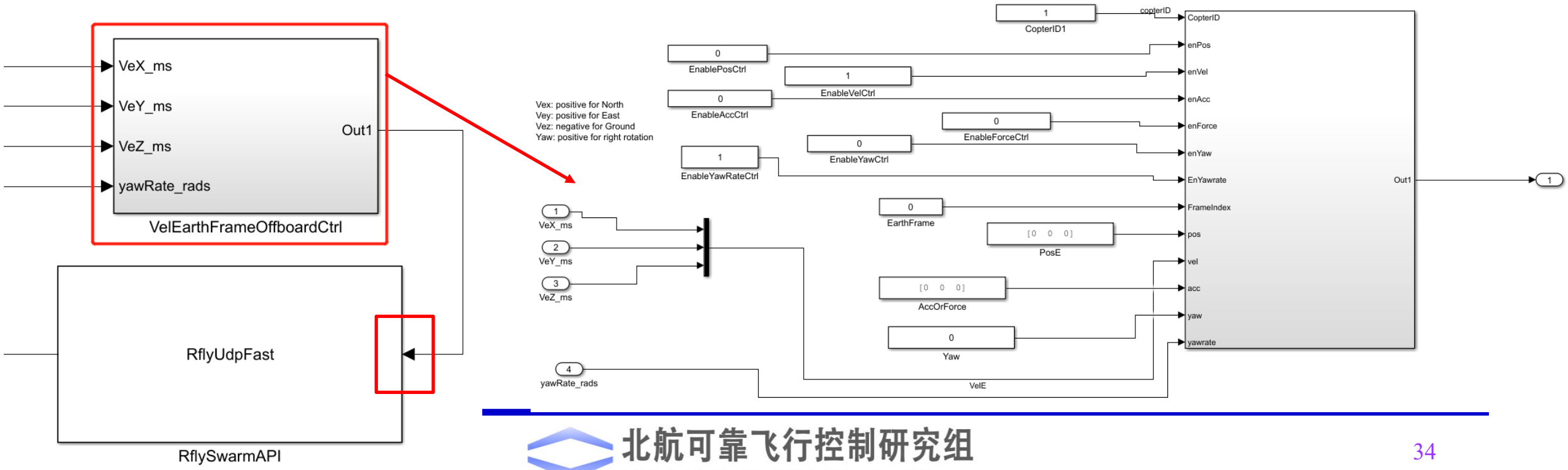
- 打开“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpFullOne.slx”即可看到RflySwarmAPI通信接口模块使用FullData模式时的例子



3.基础例子

3.2 通信接口FullData模式例子– 输入映射解析

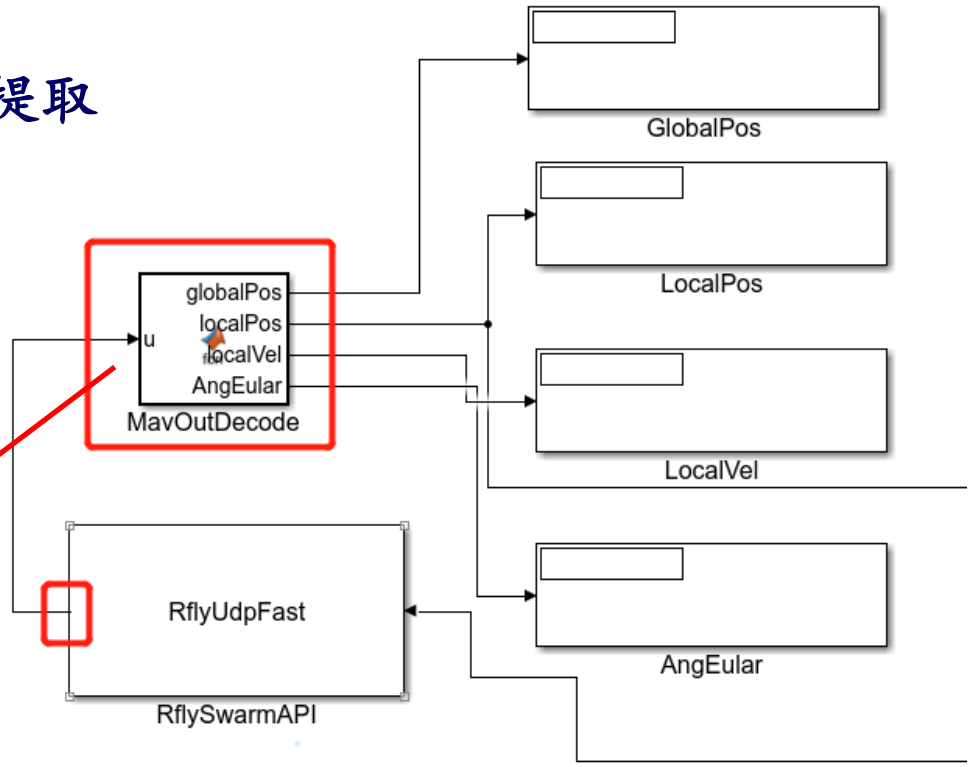
- 输入映射模块将4维的速度（地球坐标系下速度与偏航角速度控制量）转化为15维信号（见第2.4节定义）给“RflySwarmAPI”模块的FullData Mode模式。
- FullData模式可以实现所有Offboard控制功能（需要修改右下图所示的一些配置）



3.基础例子

3.3 通信接口FullData模式例子– 输出提取解析

- 将RflySwarmAPI模块的28维输出（见2.5节定义）提取出感兴趣的值: GpsHome、LocalPos、AngEular等
- 将GpsHome的经纬高数据转为CopterSim的全局坐标GlobalPos的xyz单位m的数据。代码如下

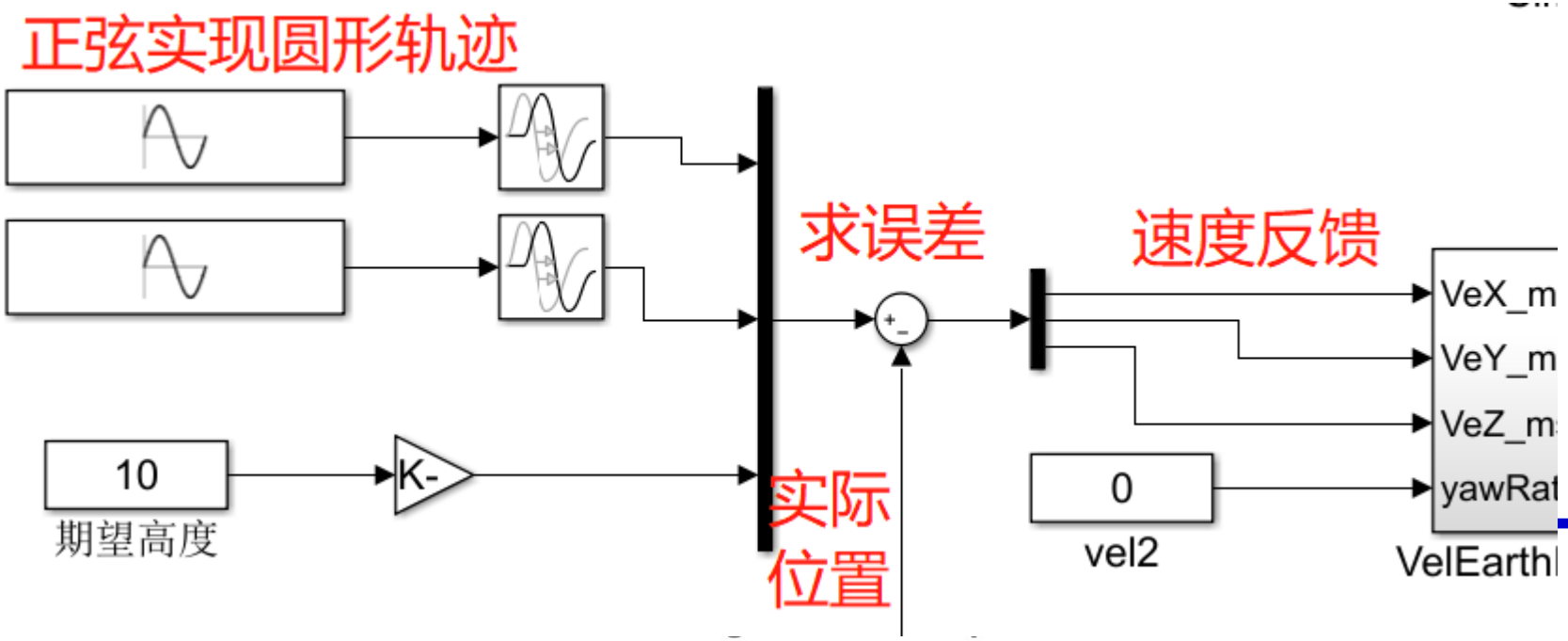



```
function [globalPos,localPos,localVel,AngEular] = fcn(u)
GpsHomePos = u(1:3);
AngEular = u(4:6);
localPos = u(7:9);
localVel = u(10:12);
globalPos = [0,0,0];
if ~(abs(GpsHomePos(1))<1&&abs(GpsHomePos(2))<1)
    globalPos(1)=(GpsHomePos(1)*1e-7-40.1540302)/180*pi*6362000+localPos(1);
    globalPos(2)=(GpsHomePos(2)*1e-7-116.2593683)/180*pi*4.8823e6+localPos(2);
    globalPos(3)=-GpsHomePos(3)*1e-3+localPos(3);
end
```

3.基础例子

3.4 通信接口FullData模式例子– 控制原理

- 轨迹生成：如左下图所示，利用正弦和余弦函数的组合形成一个原型轨迹，高度始终保持10，因此是一个定高飞圆的轨迹（这里的轨迹也可以替换成任意其它轨迹，例如8字，请自行尝试修改）。
- 期望轨迹与实时位置（LocalPos，相对解锁/起飞点的位置）做差，乘以1形成一个P控制器，将速度误差反馈回Pixhawk（只要有位置误差，速度就不为0，使两者靠拢）。



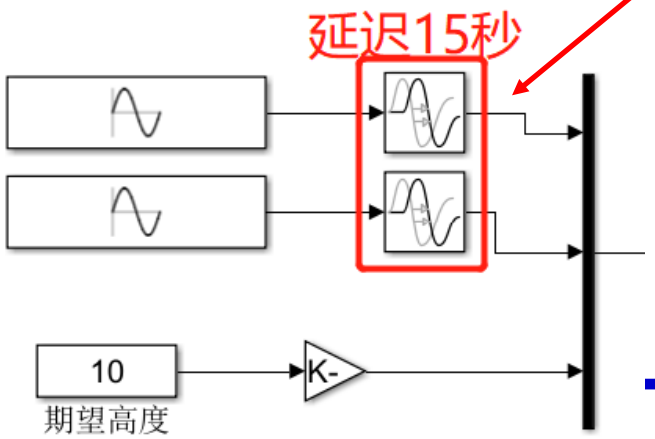
3.基础例子

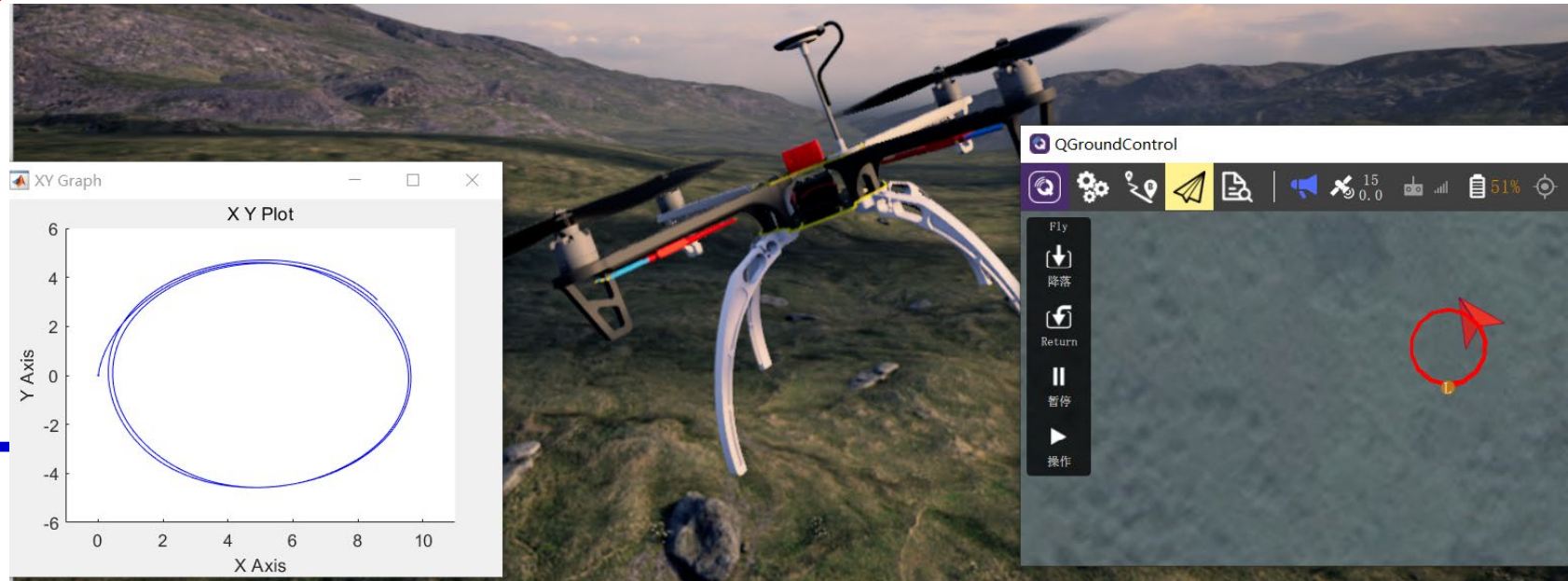
```
41  REM Set UDP data mode; 0: UDP_FULL, 1:UDP_Simple, 2: M
42  REM e.g., UDPSIMMODE=0 equals to UDPSIMMODE=UDP_Simple
43  SET UDPSIMMODE=0
```

Bat脚本

3.5 通信接口FullData模式例子– 实验现象

- 双击运行“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpFullOne.bat”（或者双击上层目录的“SITLRunUdpFull.bat”脚本，输入数字1并回车）打开单机的SITL软件在环仿真系统（或运行HITLRunUdpFull.bat 开启多机硬件在环仿真）。注意：本脚本中“UDPSIMMODE”选项需设置成“UDP_Full”，对应“RflySwarmAPI”中的数据模式。
- 用MATLAB打开“RflyUdpFullOne.slx”例程，点击运行，即可看到单机起飞到一定高度，一段时间后（正弦函数前的延迟模块）开始画圆飞行。

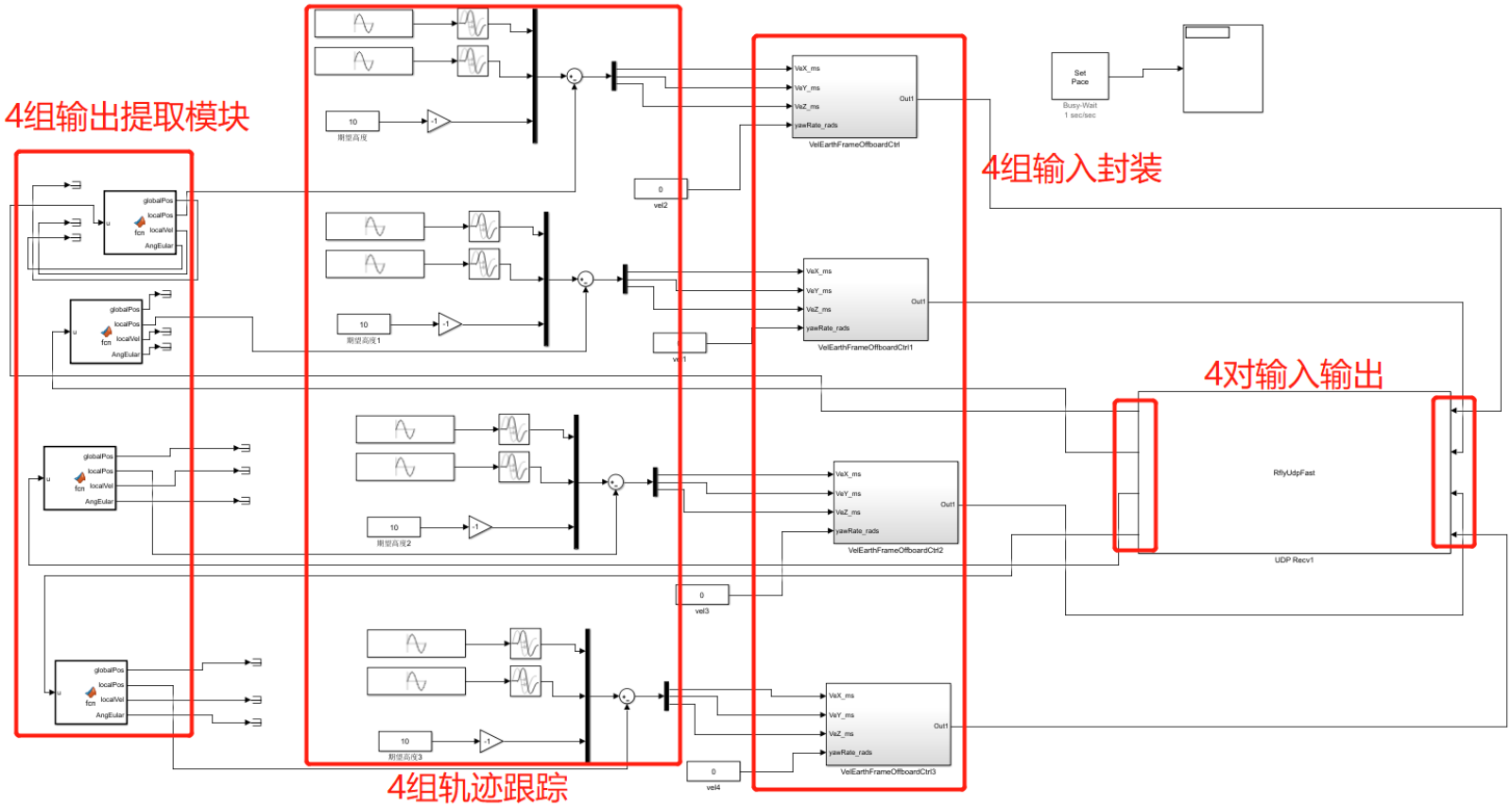




3.基础例子

3.6 通信接口FullData模式例子 – 4个飞机

- 打开“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpFullFour.slx”
- 将RflySwarmAPI模块内vehicle number设成4就能得到4对输入输出，然后每个输入/输出口复制粘贴上数据处理模块，再将控制算法复制4份即可



3.基础例子

3.7 通信接口FullData模式例子 – 4个飞机实验效果

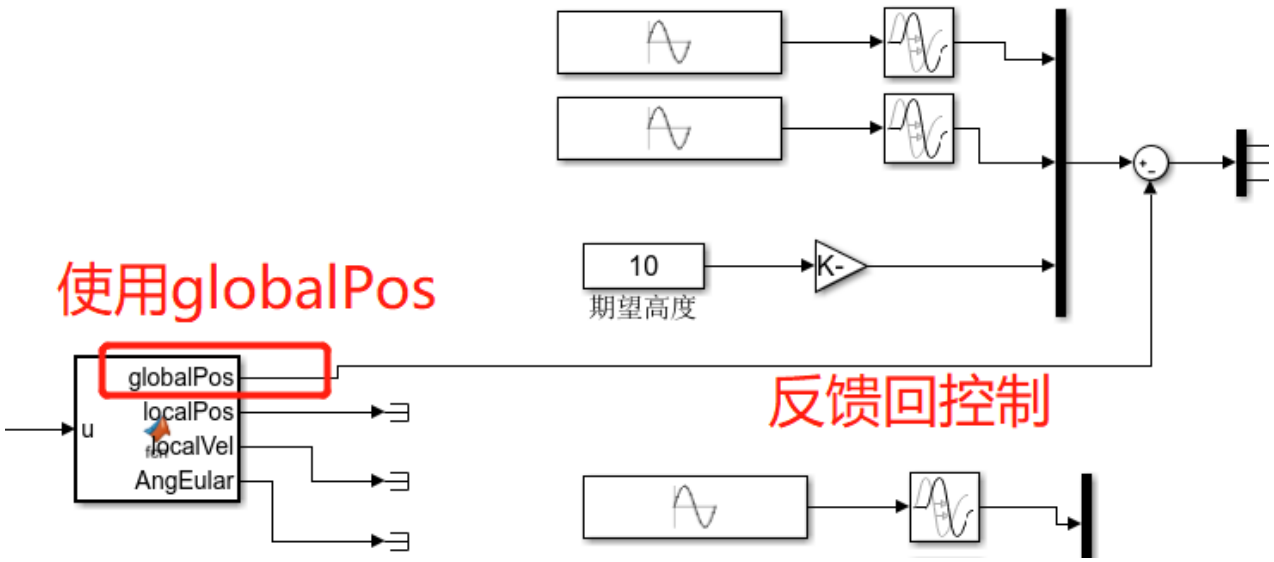
- 双击运行“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpFullFour.bat”（或者双击上层目录的“SITLRunUdpFull.bat”脚本，输入数字4并回车）打开多机的SITL软件在环仿真系统（或运行HITLRunUdpFull.bat 开启硬件在环仿真）。
- 用MATLAB打开“RflyUdpFullFour.slx”例程，点击运行，即可看到4个起飞依次到一定高度开始画圆飞行（每个飞机的圆心不同，在QGC上切换Vehicle *，查看各飞机的轨迹）。
- 注：请修改slx文件，使得各个飞机圆心一致。
  （提示：将反馈的位置信号从LocalPos改为GlobalPos）
- 注：LocalPos以各自解锁位置为原点，GlobalPos以CopterSim地图中心为原点

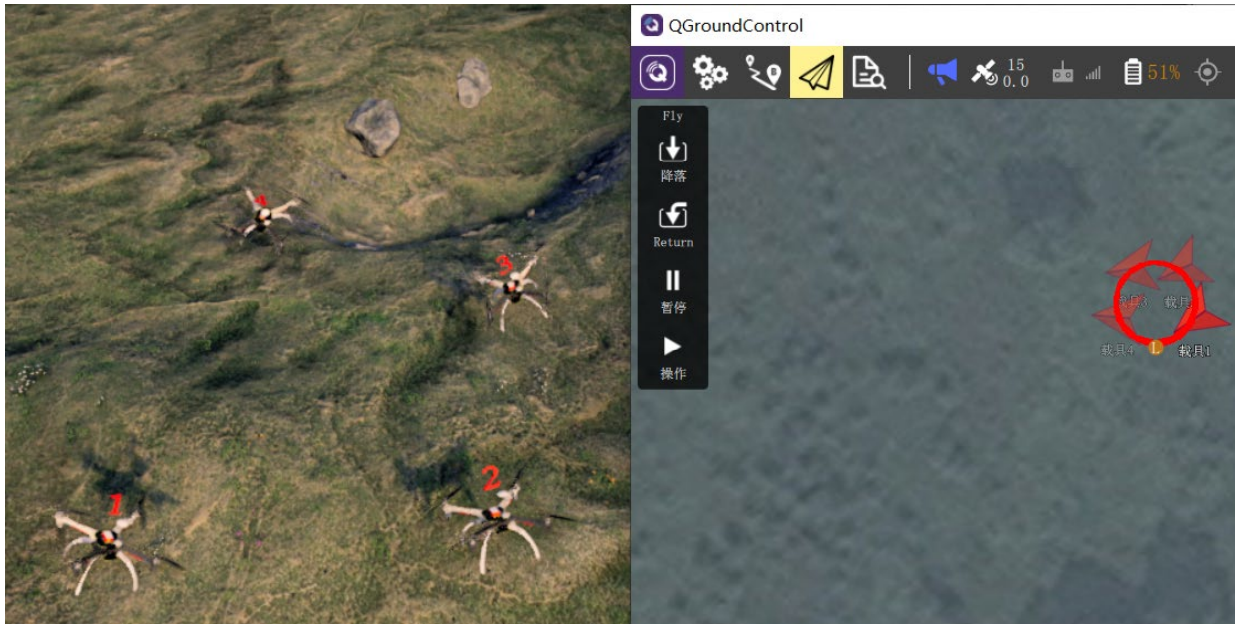


3.基础例子

3.7 通信接口FullData模式例子 – 4个飞机实验效果

- 用MATLAB打开“RflyUdpFullFour.slx”例程，按左下图修改本文件，将反馈位置从LocalPos改为GlobalPos，确保所有飞机使用CopterSim地图坐标系
- 点击“RflyUdpFullFour.bat”再运行“RflyUdpFullFour.slx”可以看到飞机依次起飞并悬停到空中同一点，然后开始绕同样圆心画圆飞行。



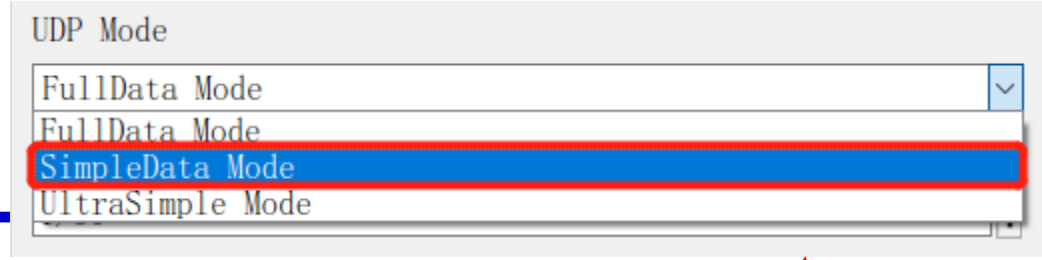


3.8 通信接口SimpleData模式例子 – 实验效果

- 双击“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\RflyUdpSimpleOne.bat”（或者双击上层目录的“SITLRunUdpSimple.bat”脚本，输入数字1并回车）打开单机的SITL软件在环仿真系统（或插入Pixhawk，运行“HITLRunUdpSimple.bat”开启单机的硬件在环仿真）。
- 用MATLAB打开“RflyUdpSimpleOne.slx”例程，点击运行，即可看到单个起飞飞机起飞到一定高度，然后一段时间后开始画圆。
- 本实验的结果与第3.5节相同，是通过两种接口来实现同样的控制功能。
- 在本例程中bat脚本中的“UDPSIMMODE”设置为了1，对应了CopterSim中“UDP Mode”设置为“UDP_Full”，也对应了RflySwarmAPI中的SimpleData模式

```
REM Set UDP data mode; 0: UDP_FULL, 1:UDP_Simple, 2: Mavlin
REM e.g., UDPSIMMODE=0 equals to UDPSIMMODE=UDP_Simple
SET UDPSIMMODE=1
```

3.基础例子

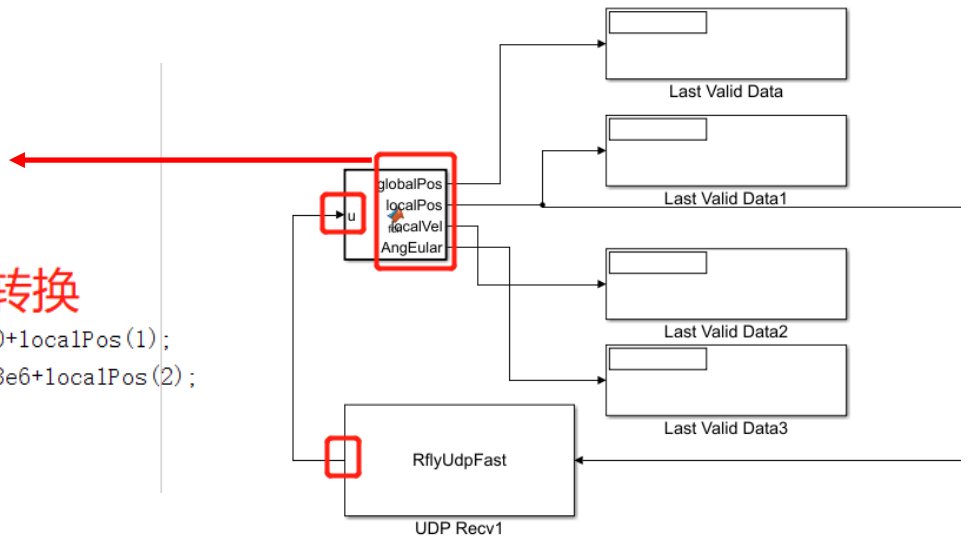


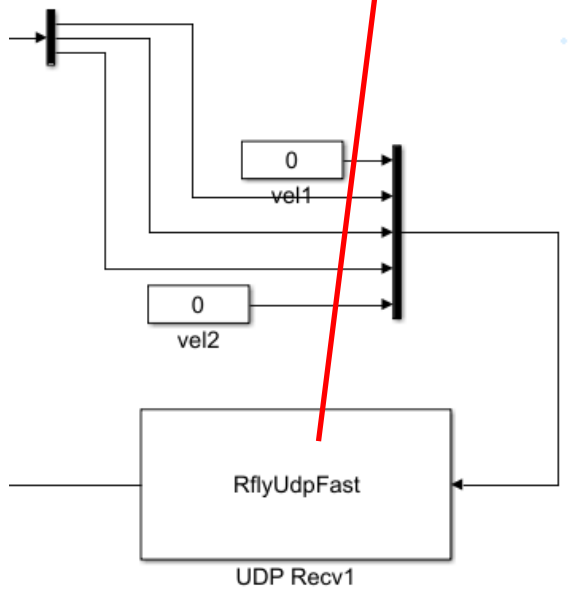
3.9 通信接口SimpleData模式例子 – 输入输出映射解析

- 如右下图所示，“RflySwarmAPI”输入部分不需要映射，直接按SimpleData的数据（见第2.5节定义）给5维的控制信号即可。
- 如左下图所示，“RflySwarmAPI”输出模块需要映射，求出CopterSim全局坐标GlobalPos的值（留作多机集群用）

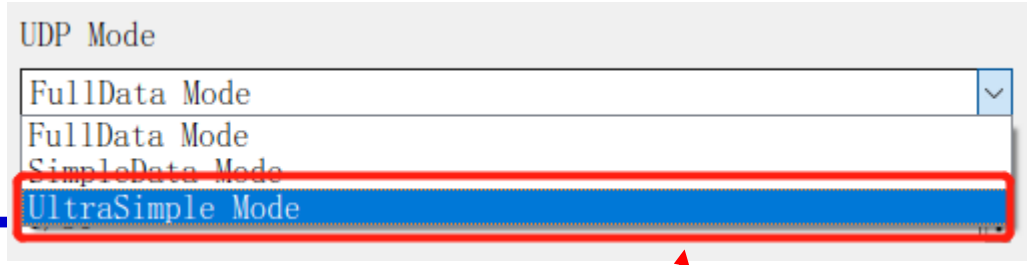
```
function [globalPos,localPos,localVel,AngEular] = fcn(u)
GpsHomePos = u(1:3);
AngEular = u(4:6);
localPos = u(7:9);
localVel = u(10:12);
globalPos = [0,0,0];
if ~(abs(GpsHomePos(1))<1&&abs(GpsHomePos(2))<1)
    globalPos(1)=(GpsHomePos(1)*1e-7-40.1540302)/180*pi*6362000+localPos(1);
    globalPos(2)=(GpsHomePos(2)*1e-7-116.2593683)/180*pi*4.8823e6+localPos(2);
    globalPos(3)=-GpsHomePos(3)*1e-3+localPos(3);
end
```

经纬高转换



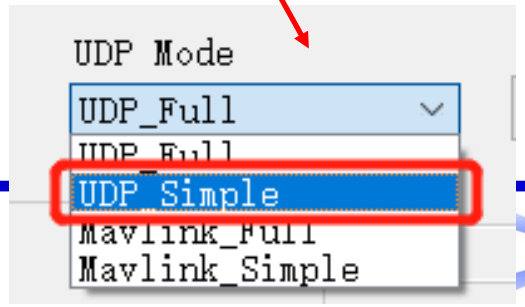


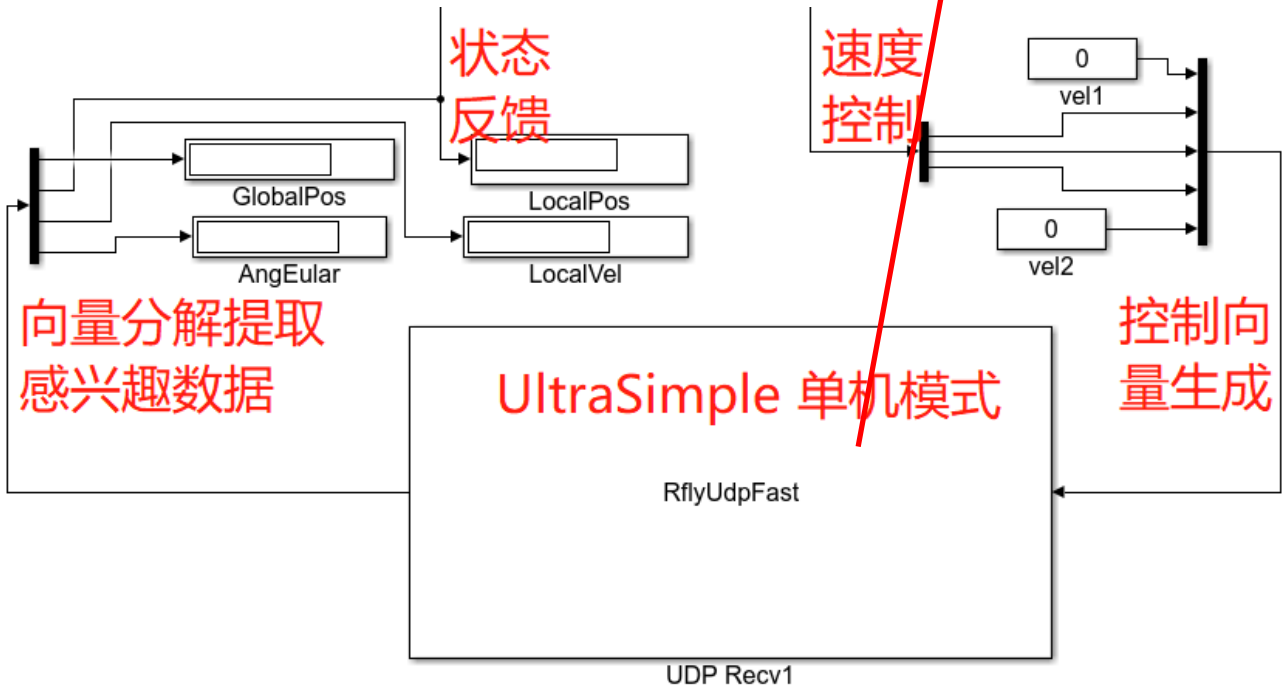
3.基础例子



3.10 通信接口UltraSimple模式例子 – 输入输出映射解析

- 打开“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\ RflyUdpUltraSimpleOne.slx”可以查看RflySwarmAPI模块使用UltraSimple模式的例子，本例子传输数据量少、延迟少，适合较多飞机的集群速度/位置/轨迹控制的的需求。
- RflySwarmAPI模块UltraSimple模式的输入为5维向量，其定义上一小节内容完全相同；输出为12维向量，其定义可以参考第2.8小节内容。
- 如右图所示，本接口不需要额外处理，直接用向量分解就能获取感兴趣的值（全局位置、本地位置、姿态等）用于控制。
- 本例子要求CopterSim使用UDP_Simple







3.基础例子

3.11通信接口UltraSimple模式例子 – 实验效果

- 进入“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo”文件夹
- 双击“RflyUdpUltraSimpleOne.bat”(或者双击上层目录的“SITLRunUdpSimple.bat”脚本，输入数字1并回车)再运行“RflyUdpUltraSimpleOne.slx”可以看到一个飞机起飞并画圆。
- 双击“RflyUdpUltraSimpleFour.bat”(或者双击上层目录的“RflyUdpUltraSimpleFour.bat”脚本，输入数字4并回车)再运行“RflyUdpUltraSimpleFour.slx”可以看到4个飞机起飞并画圆。
- 双击“RflyUdpUltraSimpleEight.bat”(或者双击上层目录的“SITLRunUdpSimple.bat”脚本，输入数字8并回车)再运行“RflyUdpUltraSimpleEight.slx”可以看到8个飞机起飞并画圆。（飞行效果与电脑配置相关，需要较高CPU和GPU）
- 以上例程也可以插入特定数量的Pixhawk，然后运行“HITLRunUdpSimple.bat”并输入串口号字符串来开启多机硬件在环仿真，并运行对应的slx文件观察效果。

RflySim：如何通过Simulink控制软件在环仿真模式下的多无人机集群

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NTEwOA==.html

YouTube： https://youtu.be/AMZNuAtRp2w



大纲

1. 安装配置
2. 接口介绍
3. 基础例子
4. 进阶例子
5. 小结

本讲的例程源码路径：
RflySimAPIs\SimulinkSwarmAPI

4.进阶例子



4.1 将Simulink集群算法编译成exe加快运行

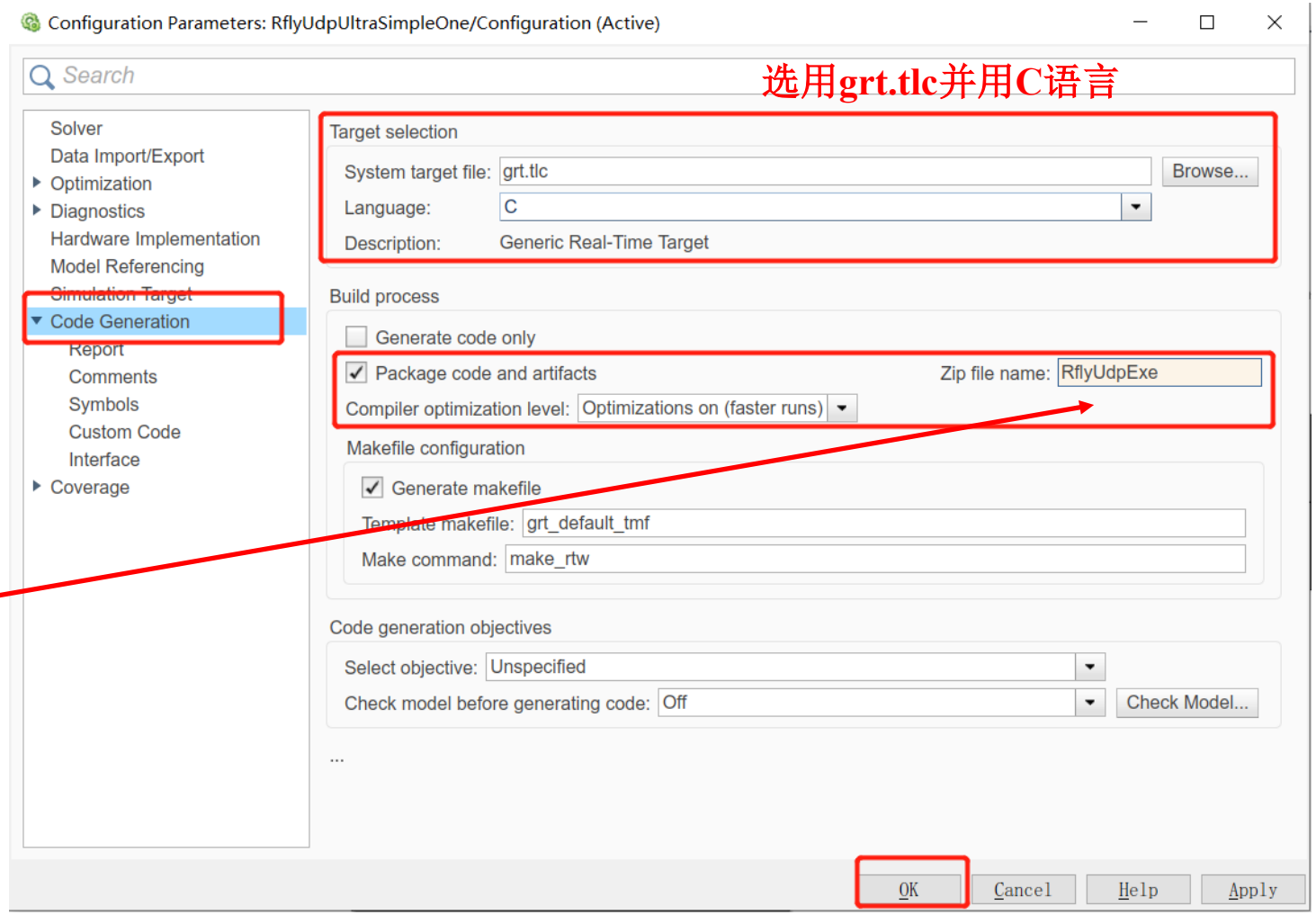
- MATLAB本身会占用大量的CPU和内存资源（见右图），在运行复杂的Simulink控制程序时，
- 一方面计算量太大导致算法运行缓慢，无法达到实时要求（Simulink中运行1s中大于现实时钟1s），这样就无法实时控制仿真系统（或真实系统）的集群飞机
- 第二方面，在仿真时Simulink如果占用大量的计算资源，会导致RflySim3D和CopterSim的计算资源分配较少，导致飞机仿真变差，飞机剧烈抖动甚至坠机。
- 将Simulink控制器编译生成exe之后，算法可以脱离MATLAB运行，而且本身是二进制可执行文件，运行效率非常高，即使大型的控制算法，也能保证实时控制。
- 如右下图所示为用exe模式运行8个飞机集群控制算法的资源管理器图片，可以看到其内存和CPU的占用几乎为0，相比Simulink直接运行有巨大的性能提升
- 注：生成exe需要安装VS编译器。



4.进阶例子

4.2 生成exe程序Simulink设置

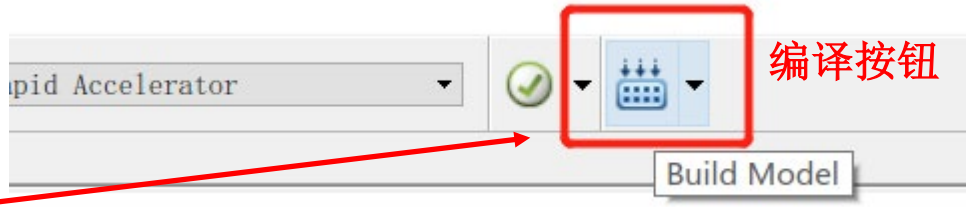
- 打开“RflyUdpUltraSimpleFour.slx”文件（或任意新建一个slx文件，但要保证GenerateSwarmExe.p、RflyUdpFast.cpp和RflyUdpFast.mexw64拷贝到同一目录）
- 点击Simulink的设置按钮，按下图设置。其他的都使用默认即可，关键是勾选“Package Code and …”，输入名字为：“RflyUdpExe”；然后，设置编译模式为：“faster runs”（这一项非必须，设置后可以使运行速度更快）。

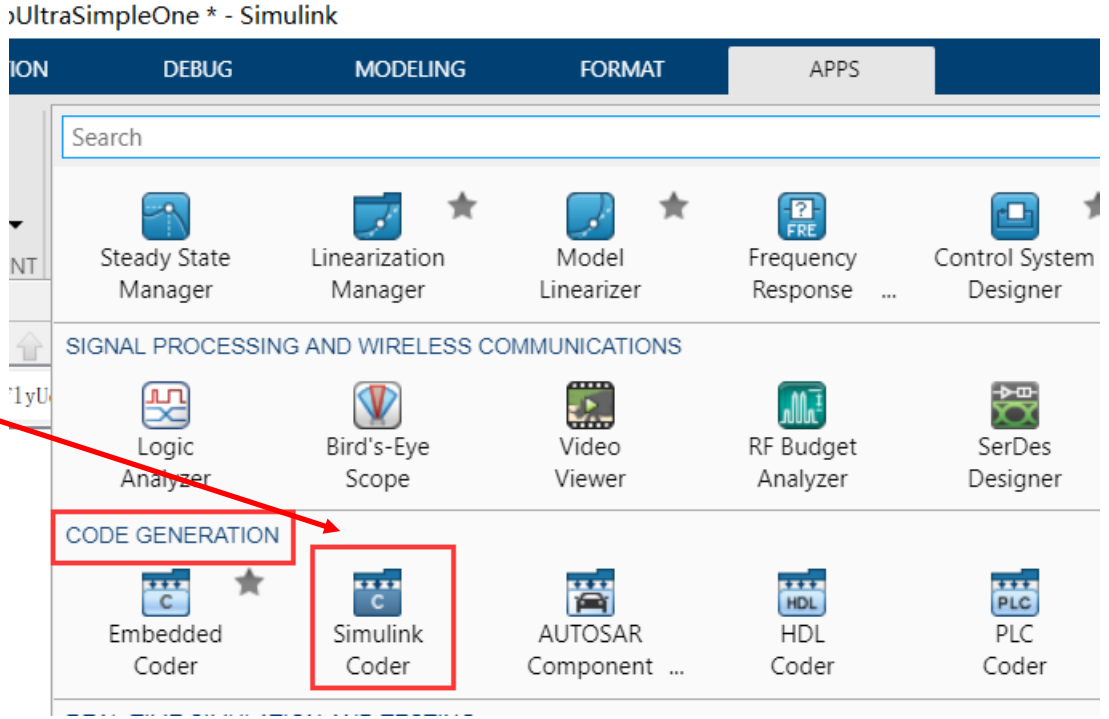


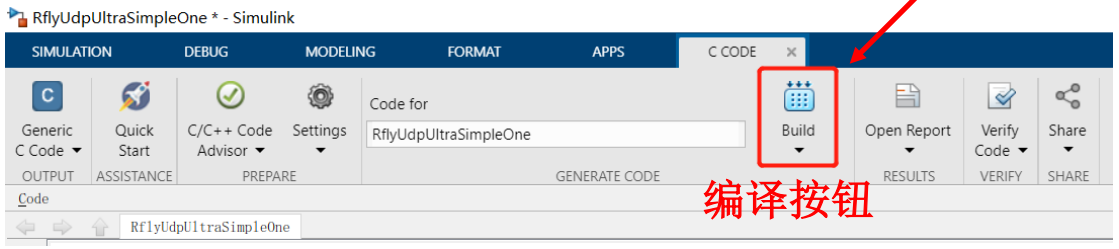
4.进阶例子

4.2 生成exe程序Simulink设置

- 点击Simulink的编译按钮
- 对于2019a及之前版本，工具栏样式见右上图，直接点击它的编译按钮即可。
- 对于对于2019b及之后版本，如右图所示，点击APP - CODE GENERATION – Simulink Coder才能弹出代码生成工具栏，在其中如下图所示点击“Build”按钮就能编译生成代码





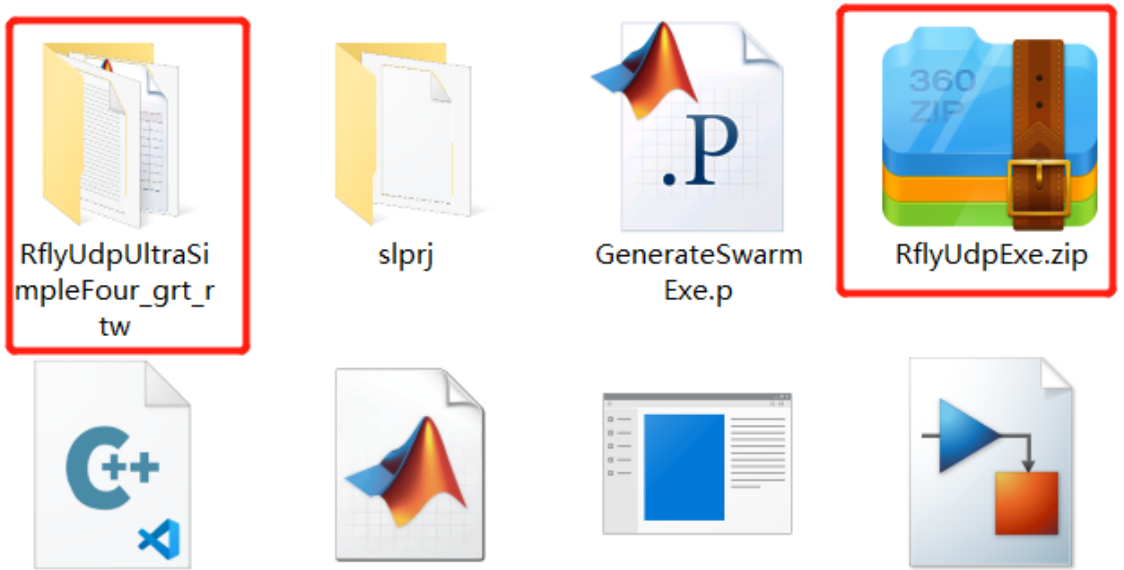


4.进阶例子

4.3生成exe程序编译结果

- 编译完成后，在“诊断视图”（Diagnostic Viewer）中得到左下图的编译成功完成提示（请忽略黄色的警告）
- 如右下图，我们可以得到一个“RflyUdpExe.zip”压缩包文件，里面包含所有生成的代码，会用于我们后续生成exe文件。

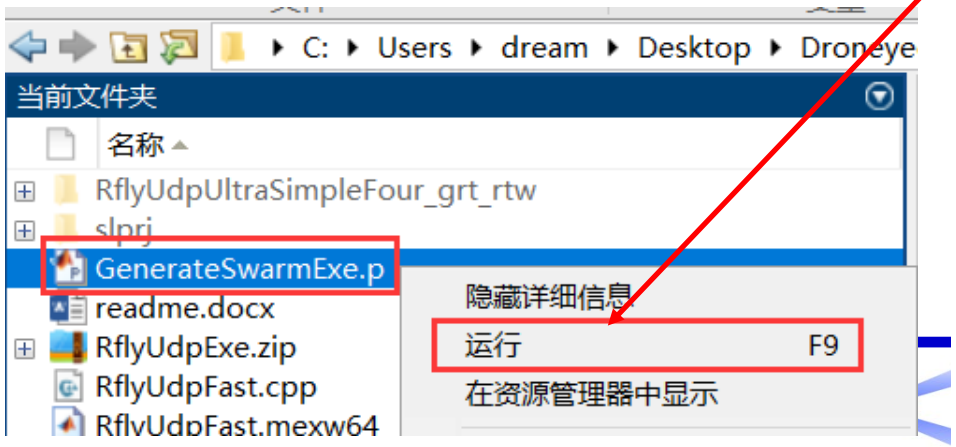


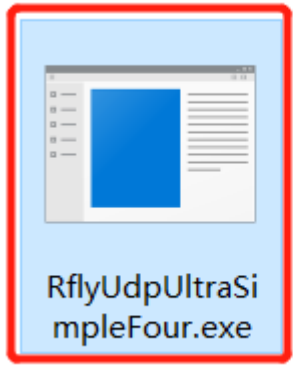


4.进阶例子

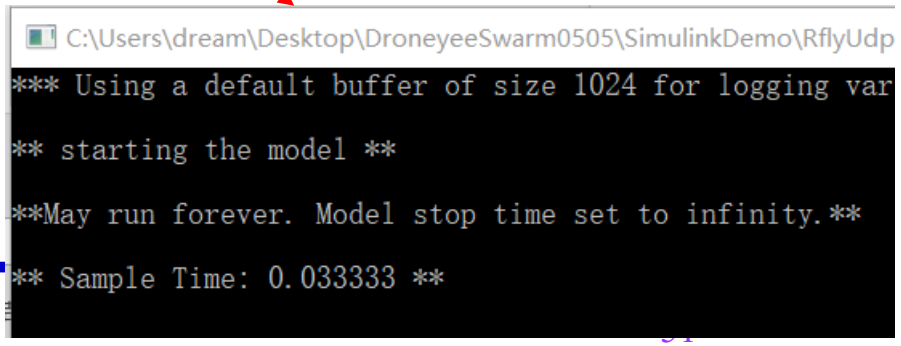
4.3 生成exe程序命令及效果

- 注意：上一步编译命令时生成了一个“RflyUdpUltraSimpleFour.exe”文件，双击它可以运行，但是它不符合实时控制要求（始终以最快速度运行，不和现实时钟同步），因此需要运行我们的脚本来生成实时控制的exe文件。
- 如下图，鼠标右键点击GenerateSwarmExe.p文件，并点击“运行。或者直接在MATLAB命令行输入“GenerateSwarmExe”，即可如下图所示得到Simulink文件名对应的.exe文件“RflyUdpUltraSimpleFour.exe”。
- 双击“RflyUdpUltraSimpleFour.bat”，开启4机SITL系统，点击上一步生成得到的exe文件即可同Simulink一样控制飞机起飞画圆，exe运行窗口如右下图。





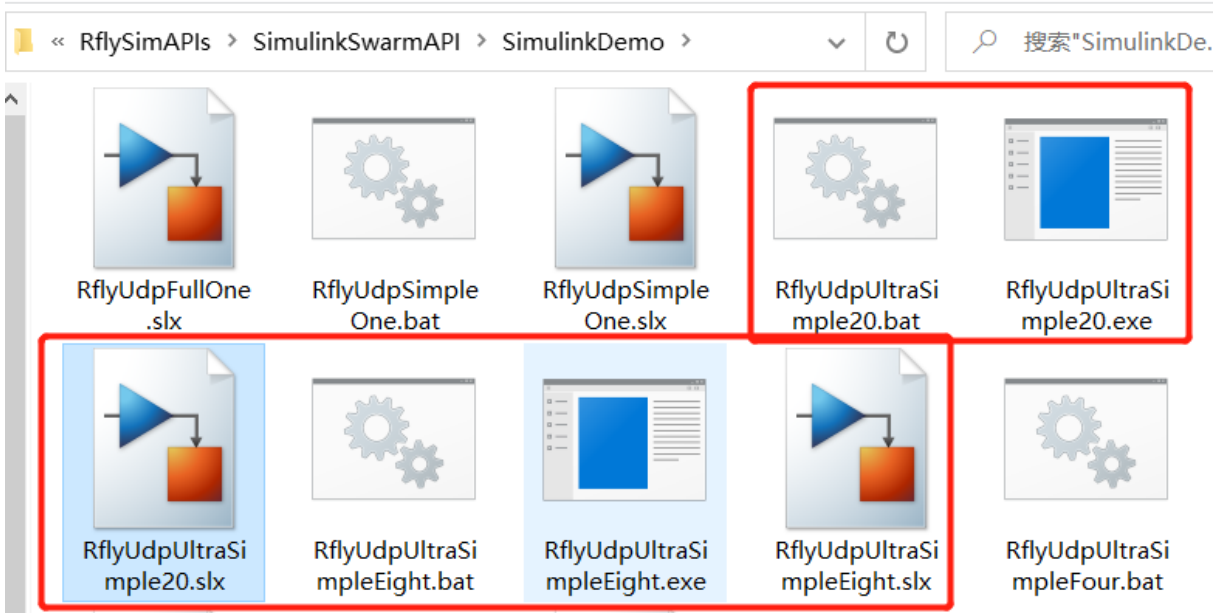




4.进阶例子

4.3 生成exe程序其他例子

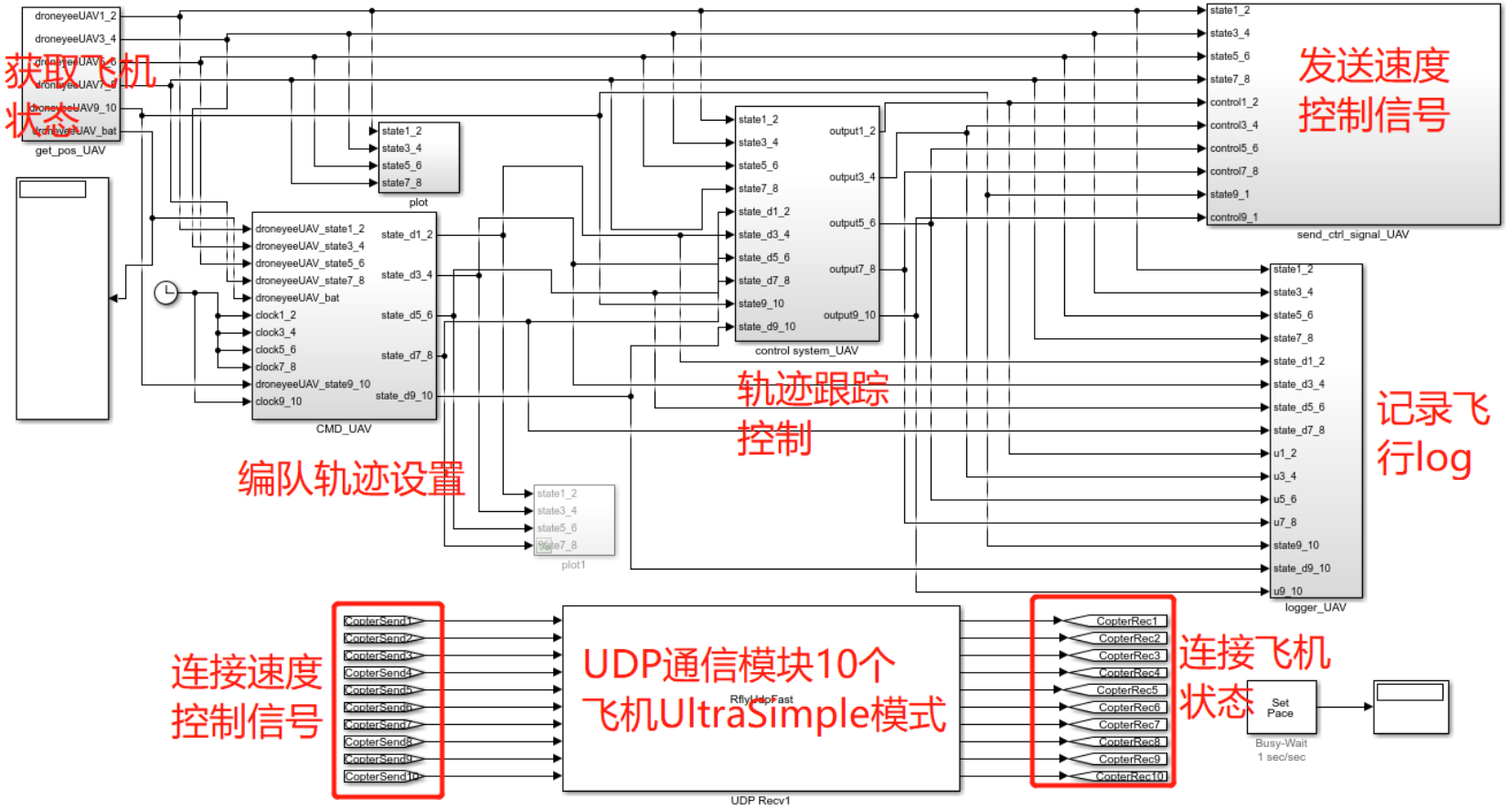
- 请自行用exe代码生成的方式运行8机飞行的例子（需要电脑配置支持），例程文件见“RflyUdpUltraSimpleEight.exe”。
- 请自行用exe代码生成的方式运行20机飞行的例子（仅限完整版，且需要电脑配置支持），例程文件见“RflyUdpUltraSimple20.exe”。
- “RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo”文件夹下所有例程均支持生成exe文件，再进行软件/硬件在环仿真。
  请自行尝试运行这些例子。
- 注：视觉版和集群版限制了最大飞机数量为8个，因此无法运行20飞机的例程



4.进阶例子

4.4 八个飞机飞8字编队例子

- 进入“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\UAV8Swarm3D”文件夹
- 用MATLAB打开“UAV8Swarm3D.slx”文件，可以看到其中的Simulink飞8字编队控制算法。注：本算法可以用于1~10个飞机的编队控制。
- 自行阅读内部实现



4.进阶例子

4.5 八个飞机飞8字编队例子 – 实验效果

- 运行本目录的“UAV8Swarm3D.bat”会自动打开八个飞机的SITL仿真模式。或者插入8个飞控，运行“RflySimAPIs\SimulinkSwarmAPI\HITLRunUdpSimple.bat”来开启八个飞机的硬件在环。
- 注：本例子最多可控制10个飞机的编队，也可以将“UAV8Swarm3D.bat”修改VehicleNum 幅值语句为“SET VehicleNum=10”。
- 打开MATLAB，运行“UAV8Swarm3D.slx”文件即可看到飞机依次起飞，编队飞8字。注意：配置不够的电脑，同时运行8个飞机SITL和MATLAB可能会卡顿而炸机，也可以不打开MATLAB，按之前步骤生成“UAV8Swarm3D.exe”来运行8字编队。



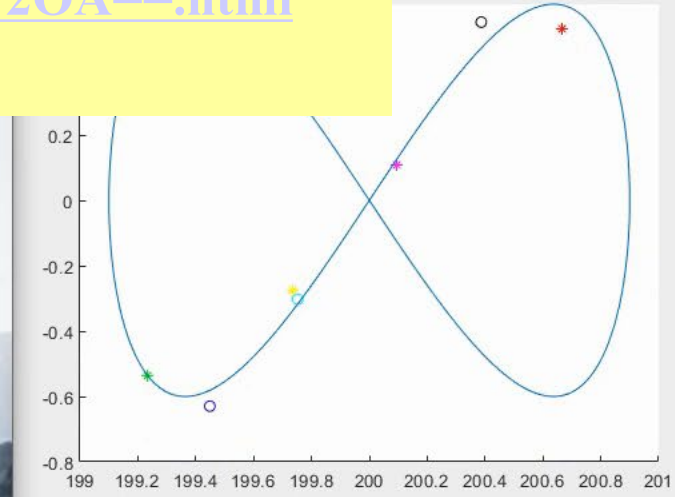
RflySim: 如何快速将集群仿真算法部署到室内真机集群平台上

本视频观看地址：

优酷：https://v.youku.com/v_show/id_XNDcwNjA4NDU2OA==.html

YouTube：https://youtu.be/sLIatdHL6FY





在RflySim仿真平台中利用软件/硬件在环实现八个飞机8字编队验证

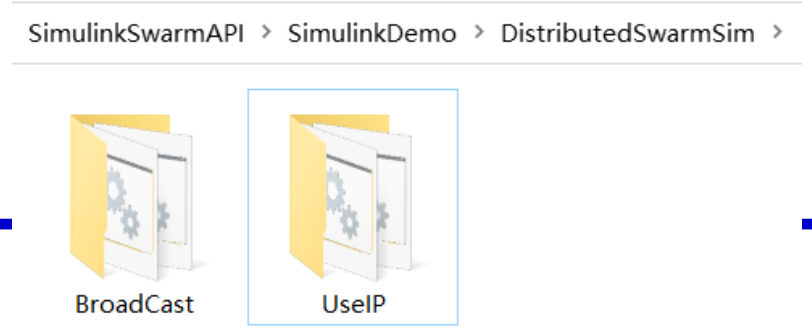
4.进阶例子

如右图所示，本例子的代码还可以直接用于更多飞机的编队实验。

注：室内试验需要用到Optitrack视觉定位系统来获取位置，室外可直接用GPS定位来编队。

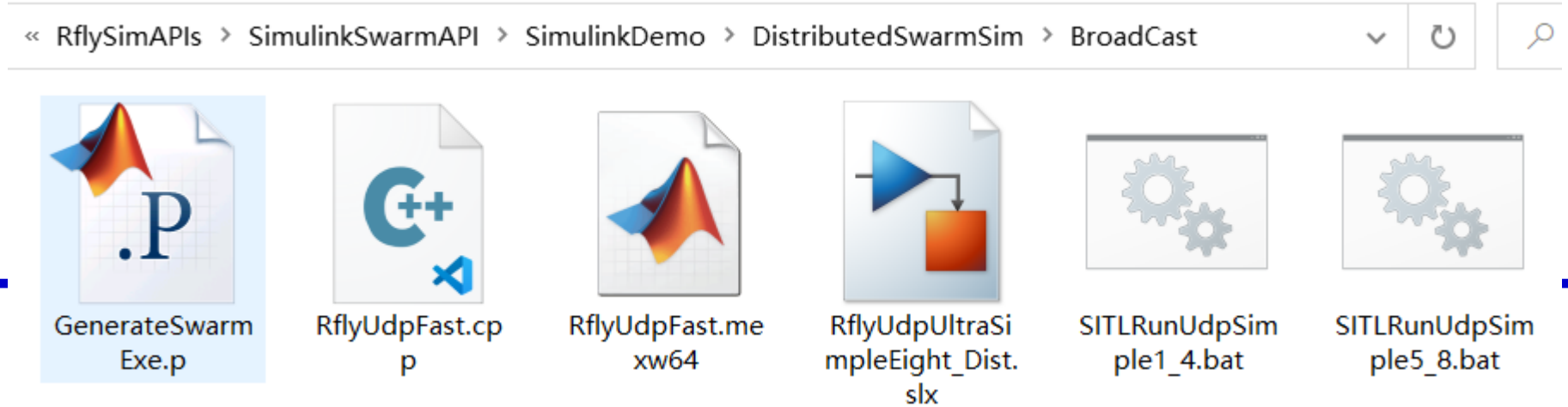


4.进阶例子



4.6 分布式局域网仿真（仅限完整版）

- 例程文件见“RflySimAPIs\SimulinkSwarmAPI\SimulinkDemo\DistributedSwarmSim”，其中包含了两个文件夹“BroadCast”：用广播的通信方式来局域网组网仿真；“UseIP”：用指定两台电脑IP地址的方式来组网仿真。
- “BroadCast”的例子的优点是配置简单，不需要查看两台电脑在局域网中的地址，拷贝到其他电脑可以直接运行，但是通信效率低、不稳定、延迟大。
- “UseIP”的例子与广播的方式相反，配置较为麻烦（需要指定两台电脑的IP），但是性能较高、通信速度快、延迟小，在更多飞机的仿真时推荐使用本模式。
- 在两个代码文件夹中都包含两个bat文件，需要在两台电脑分别运行



4.进阶例子

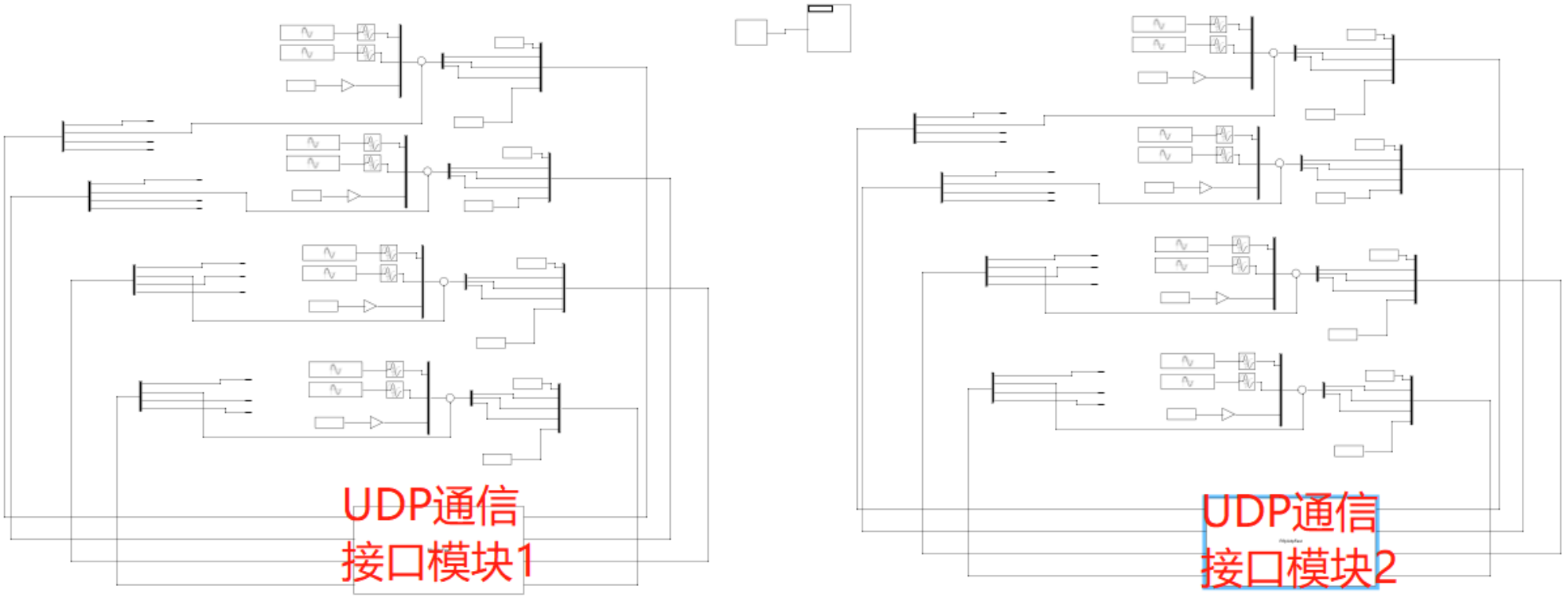
4.6 分布式局域网仿真bat脚本编写注意事项

- “SITLRunUdpSimple1_4.bat”用于一台电脑，用于开启1~4号飞机的软件在环仿真。相对于其他bat脚本，它的修改如下
- SET /a START_INDEX=1语句后，另起一行加上SET /a TOTOAL_COPTER=8；对于“BroadCast”例程，将其SET IS_BROADCAST=0 修改为：“SET IS_BROADCAST=1”；对于“UseIP”例程，使用“SET IS_BROADCAST= 192.168.1.5”，其中“192.168.1.5”是用于Simulink集群控制的电脑的IP地址。
- “SITLRunUdpSimple5_8.bat”用于第二台电脑，用于开启5~8号飞机的软件在环仿真。相对于其他bat脚本，它的修改如下
- 将其SET /a START_INDEX=1 修改为：SET /a START_INDEX=5；在该语句后，令起一行加上SET /a TOTOAL_COPTER=8；对于“BroadCast”例程，将其SET IS_BROADCAST=0 修改为：“SET IS_BROADCAST=1”；对于“UseIP”例程，使用“SET IS_BROADCAST= 192.168.1.5”，其中“192.168.1.5”是用于Simulink集群控制的电脑的IP地址。

4.进阶例子

4.6 分布式局域网仿真Simulink例子

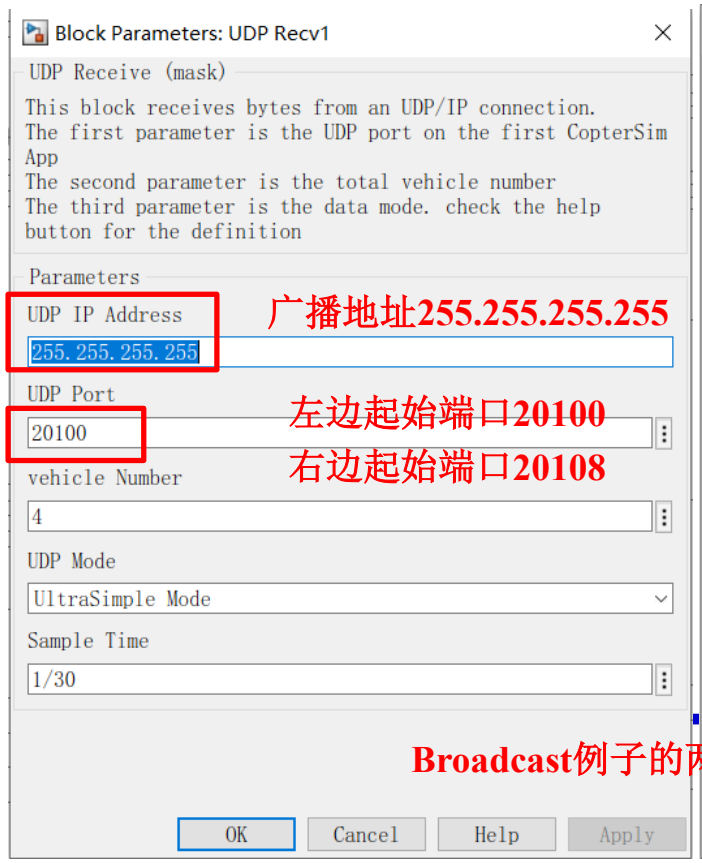
- 打开“DistributedSwarmSim\BroadCast\ RflyUdpUltraSimpleEight_Dist.slx”，器内部实现情况如下，里面包含了两个UDP通信模块，分别于两台电脑进行通信

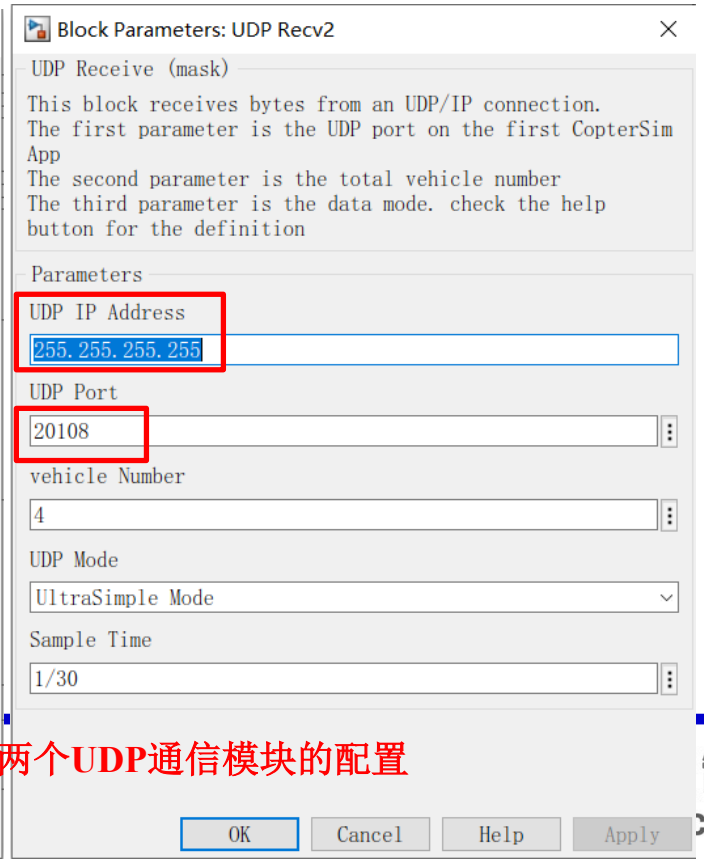


4.进阶例子

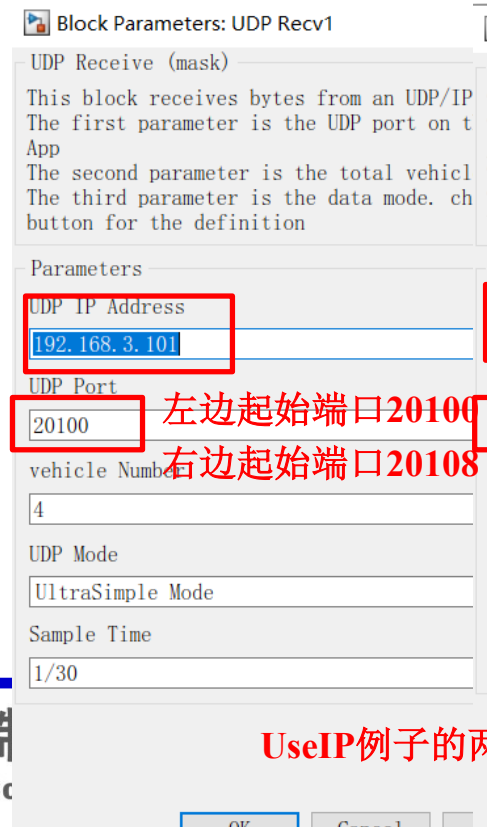
4.6 分布式局域网仿真Simulink例子

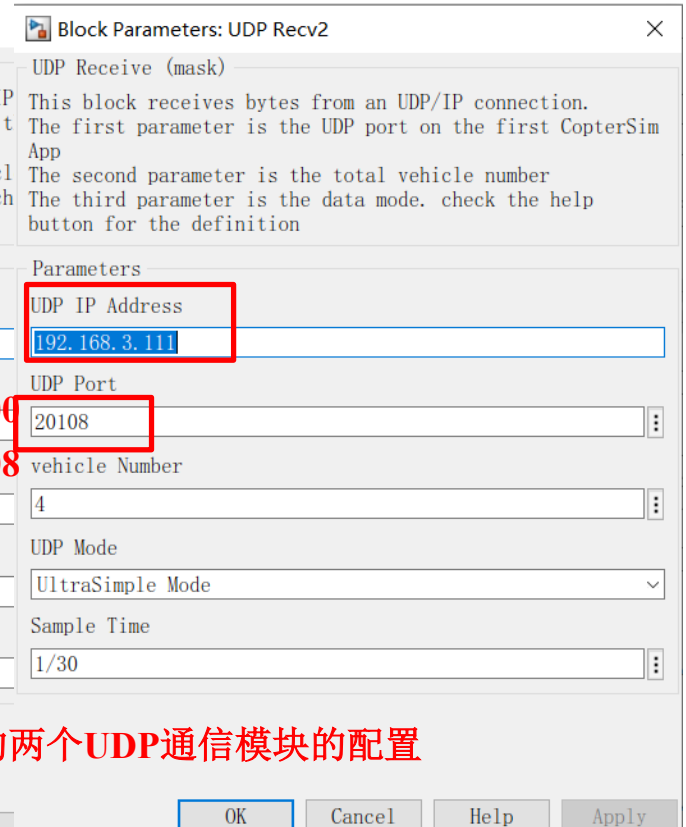
- BroadCast例子的IP地址都输入“255.255.255.255”，UseIP的例子要分别输入被控飞机所在电脑的IP地址（例如192.168.3.101）
- 左边模块对应1~4号飞机，起始端口号为20100；右边飞机从5开始，起始端口20108





Broadcast例子的两个UDP通信模块的配置





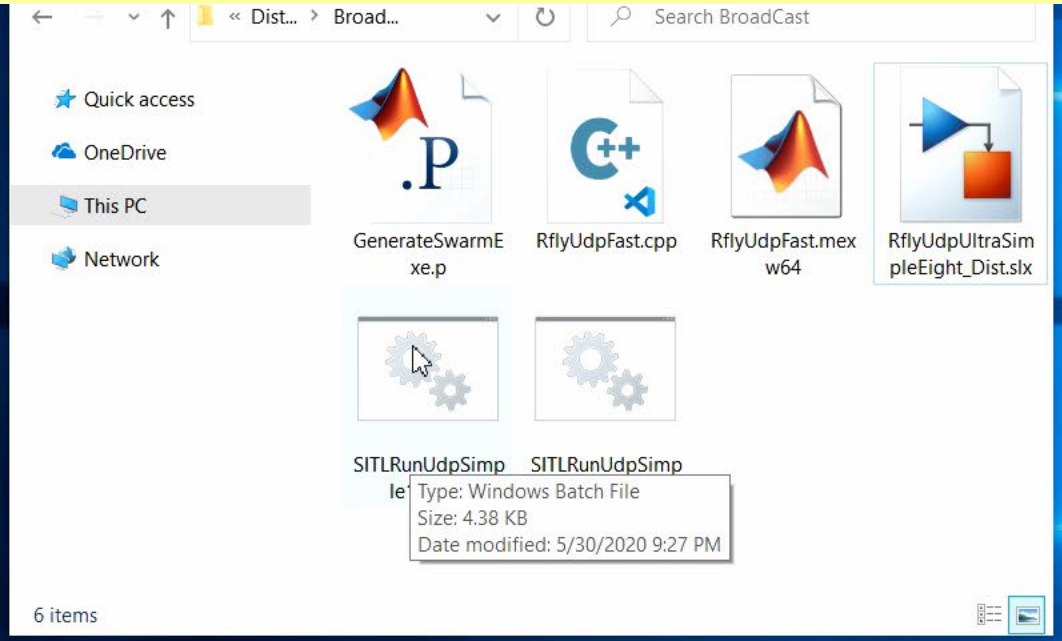
UseIP例子的两个UDP通信模块的配置

RflySim: 如何在局域网多台电脑上快速启动分布式的无人机集群软件在环仿真

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NDE2OA==.html

YouTube： https://youtu.be/fmzYADSQyj0

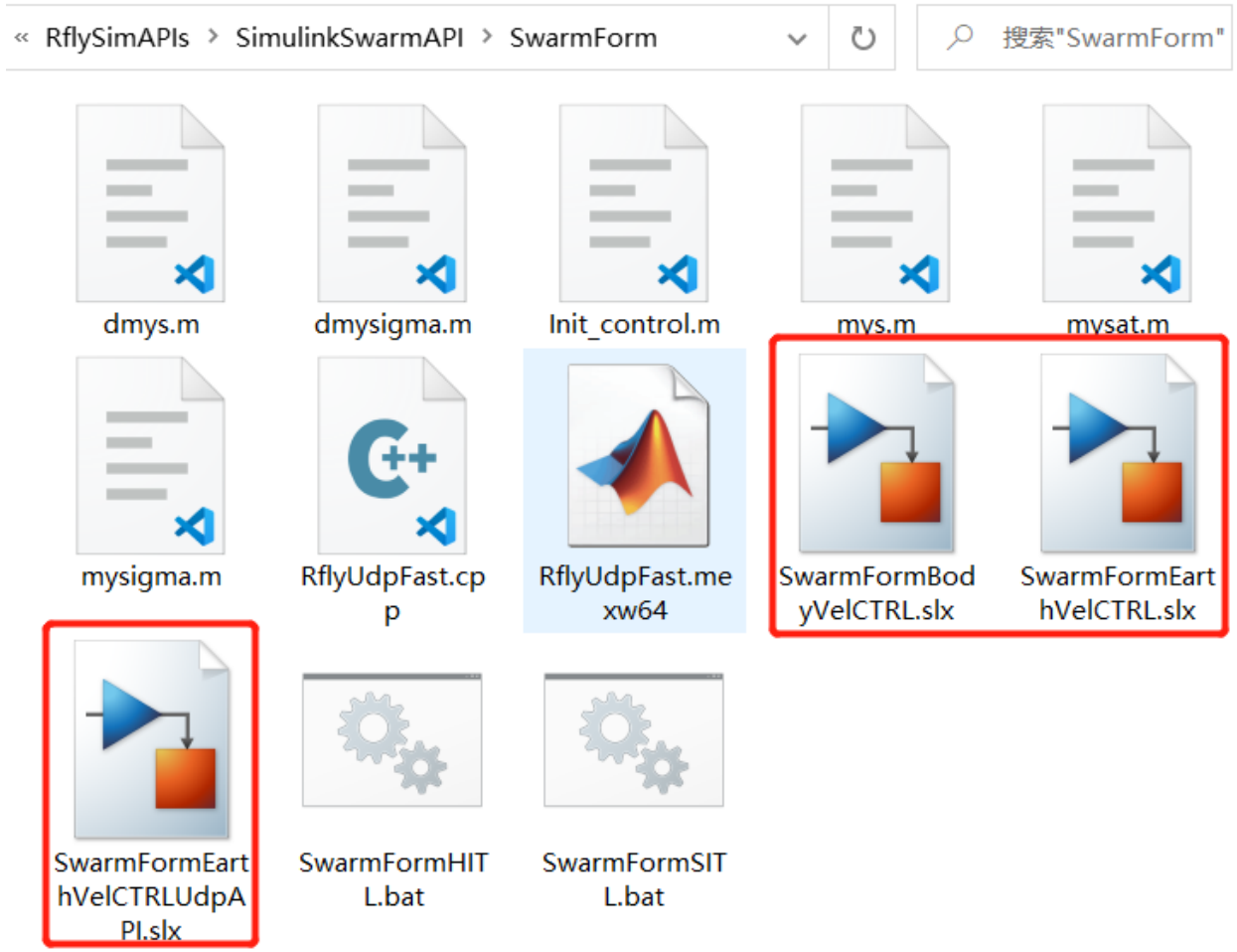


ENG
10:03 PM
5/30/2020

4.进阶例子

4.7 带自动防撞的无编队集群算法实现

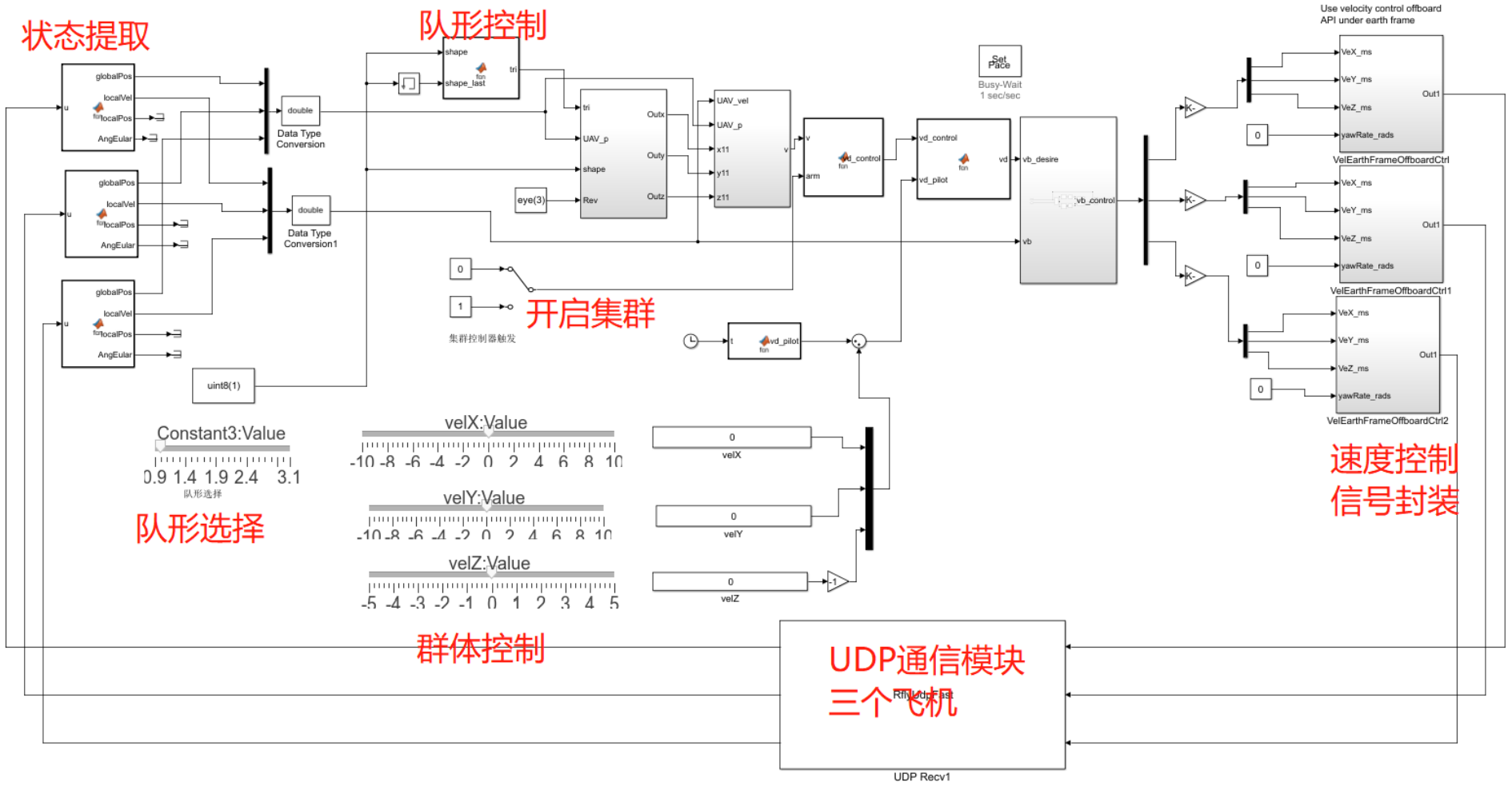
- 实验例程见文件夹“RflySimAPIs\SimulinkSwarmAPI\SwarmForm”，其中包含三个程序：
- “SwarmFormBodyVelCTRL.slx”是用Simulnk的UDP模块通过发送机体速度信号来控制飞机编队
- “SwarmFormEarthVelCTRL.slx” 用Simulnk的UDP模块通过发送地球速度（NED坐标系）信号来控制飞机编队
- “SwarmFormEarthVelCTRLUdpAPI.slx”是利用我们提供的多机UDP模块（UDP_Full模式）来发送地球速度（NED坐标系）控制信号。



4.进阶例子

4.7 带自动防撞的无编队集群算法实现

- 下面以“SwarmFormEarthVelCTRLUdpAPI.slx”为例子，介绍一下操作流程



4.进阶例子

4.7 带自动防撞的无编队集群算法实现 – 数学原理

- 利用人工势场法进行避障

$$\sigma(x,d_1,d_2)=\begin{cases}1 & \text{当}x\le d_1\text{时}\\ Ax^3+Bx^2+Cx+D & \text{当}d_1\le x\le d_2\text{时}\\ 0 & \text{当}d_2\le x\text{时}\end{cases}\qquad s(x,r_s)=\begin{cases}x & 0\le x\le x_1\\ (1-r_s)+\sqrt{r_s^2-(x-x_2)^2} & x_1\le x\le x_2\\ 1 & x_2\le x\end{cases}$$

$$V_{m,il}=\frac{k_3\sigma_{m,il}(\|\tilde{\xi}_{m,il}\|,2r_m,r_a)}{(1+\varepsilon)\|\tilde{\xi}_{m,il}\|-2r_m s(\frac{\|\tilde{\xi}_{m,il}\|}{2r_m},r_s)}\qquad b_{il}=-\frac{\partial V_{m,il}}{\partial\|\tilde{\xi}_{m,il}\|}\frac{1}{\|\tilde{\xi}_{m,il}\|}$$

- 人工势场速度指令表示为

$$\mathbf{V}_\mathbf{i}^{\mathbf{col}}=-\sum_{l\neq i}b_{il}\tilde{\xi}_{m,il}$$

- 响应遥控器的相对位置跟踪为

$$\mathbf{V}_\mathbf{i}^{\mathbf{pos}}=k_p\left(\mathbf{p}_\mathbf{i}-\mathbf{p}_\mathbf{i}^\mathbf{d}\right)+\frac{1}{l}\mathbf{v}_\mathbf{i}+\sum_{j=1}^{N}k_{ij}\left[\left(\mathbf{p}_\mathbf{i}-\mathbf{p}_\mathbf{i}^\mathbf{d}\right)-\left(\mathbf{p}_\mathbf{j}-\mathbf{p}_\mathbf{j}^\mathbf{d}\right)\right]$$

- 因此总的速度指令可表示为

$$\mathbf{V}_\mathbf{i}=\mathbf{V}_\mathbf{i}^{\mathbf{pos}}+\mathbf{V}_\mathbf{i}^{\mathbf{col}}=k_p\left(\mathbf{p}_\mathbf{i}-\mathbf{p}_\mathbf{i}^\mathbf{d}\right)+\frac{1}{l}\mathbf{v}_\mathbf{i}+\sum_{j=1}^{N}k_{ij}\left[\left(\mathbf{p}_\mathbf{i}-\mathbf{p}_\mathbf{i}^\mathbf{d}\right)-\left(\mathbf{p}_\mathbf{j}-\mathbf{p}_\mathbf{j}^\mathbf{d}\right)\right]-\sum_{l\neq i}b_{il}\tilde{\xi}_{m,il}$$

4.进阶例子

4.8 带自动防撞的无编队集群算法实现 – 实验效果

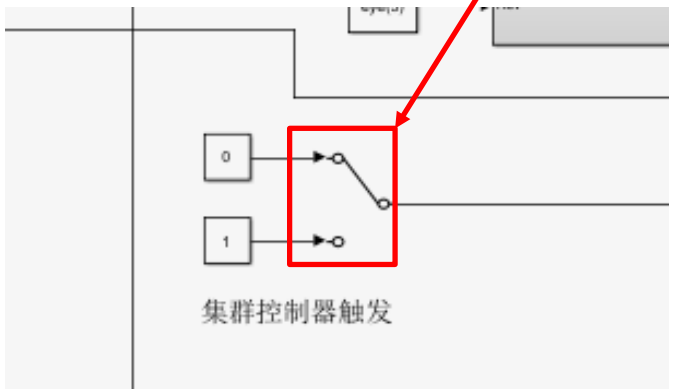
- 双击运行“SwarmFormSITL.bat”脚本来新建三个飞机的SITL仿真（或者插入三个Pixhawk，双击运行“SwarmFormHITL.bat”并输入三个飞控的串口号，来新建三个HITL仿真系统）。
- 等到CopterSim上提示EKF初始化完毕（硬件在环时Pixhawk的LED灯变绿灯闪烁，软件在环时地面站上显示进入Hold模式）
- 此时运行“SwarmFormEarthVelCTRLUdpAPI.slx”程序，可以在RflySim3D上观察到飞机慢慢起飞（按下键盘“S”键可以观看每个飞机ID）



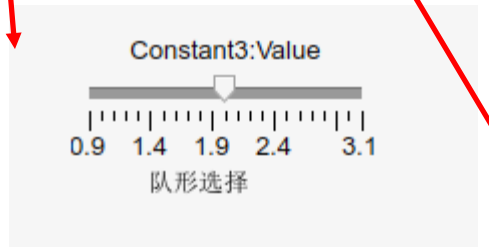
4.进阶例子

4.8 带自动防撞的无编队集群算法实现 – 实验效果

- 将其中的“集群控制器触发”开关拨到“1”，可以看到飞机组成竖直“1”字编队
- 将“队形选择”摇杆拨到中间，可以看到飞机变换队形编程“横一”字。注意，这里的原理是给定了最终队形“一”，飞机通过人工势场等算法自动飞过去，并进行避障，不需要规定每个飞机的轨迹。





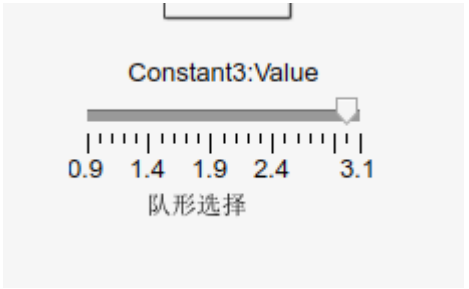


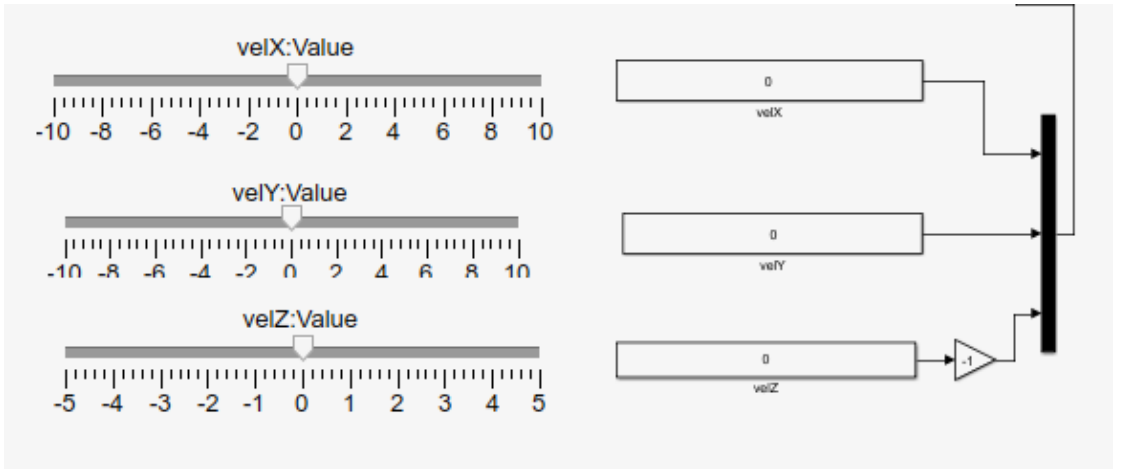


4.进阶例子

4.8 带自动防撞的无编队集群算法实现 – 实验效果

- 将“队形选择”摇杆拨到最后，可以看到飞机编程三角队形。
- 拖动“syz速度控制滑块”，就可以控制三角队形整体前后左右上下的运行。



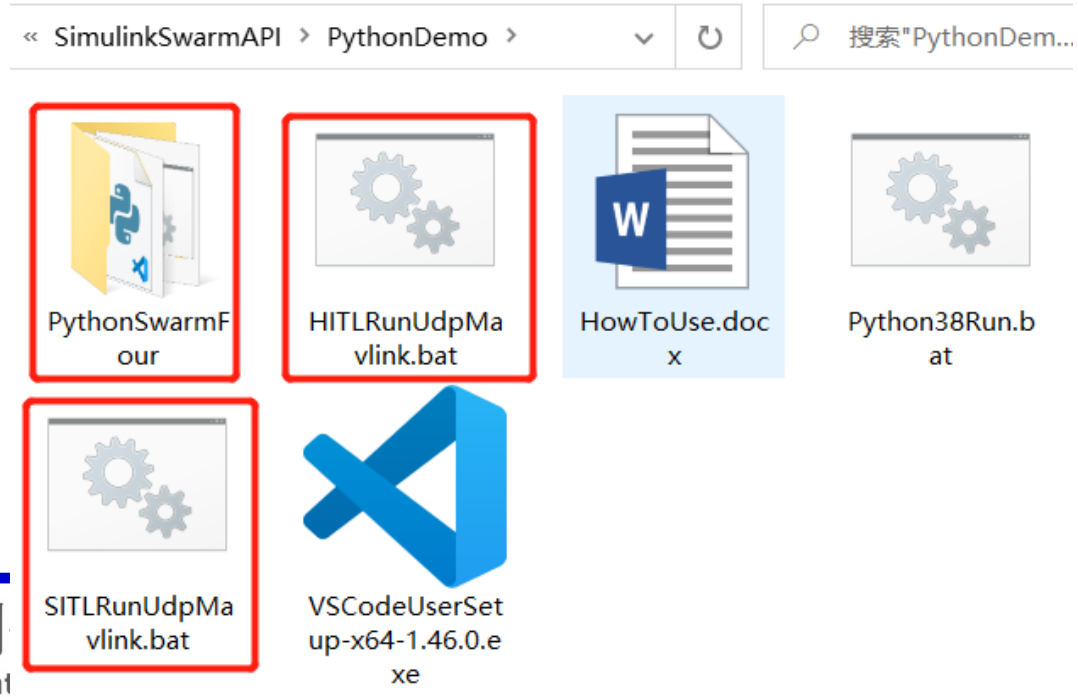




4.进阶例子

4.9 Python控制集群无人机例程

- 例程文件夹见“RflySimAPIs\SimulinkSwarmAPI\PythonDemo”，注意Python代码阅读和运行需要安装VS Code，具体安装流程请参考高级课程第6讲1.2节。
- 集群控制的Python通信接口文件为“PythonSwarmFour\PX4MavCtrlV4Swarm.py”其基本使用方法与高级课程第6讲1.2节2.5节的“PX4MavCtrlV4.py”接口高度一致，但是删去了控制RflySim3D进行视角调整等函数。
- Python控制时需要直接获取和发送飞机的Mavlink消息，因此需要使用本文件夹中的“HITLRunUdpMavlink.bat”和“SITLRunUdpMavlink.bat”来启用硬件或软件在环仿真。相对于Simulink集群控制的例子，这两个脚本主要修改了“SET UDPSIMMODE=2”，即CopterSim启用Mavlink转发模式。



4.进阶例子

4.9 Python控制集群无人机例程 – 代码解析

PythonSwarmFour.py

C: > PX4PSPFull > RflySimAPIs > SimulinkSwarmAPI > PythonDemo > PythonSw

```python
import time
import math
import sys

import PX4MavCtrlV4Swarm as PX4MavCtrl


#新建四个Mavlink通信实例，CopterSim接口是20100+2*i，i=
mav = PX4MavCtrl.PX4MavCtrler(20100)
mav1 = PX4MavCtrl.PX4MavCtrler(20102)
mav2 = PX4MavCtrl.PX4MavCtrler(20104)
mav3 = PX4MavCtrl.PX4MavCtrler(20106)

#开启Mavlink监听CopterSim数据，并更新
mav.InitMavLoop()
mav1.InitMavLoop()
mav2.InitMavLoop()
mav3.InitMavLoop()
time.sleep(2)

#显示从CopterSim收到的位置、速度、姿态角、角速度信息
print((mav.uavPosNED,mav.uavVelNED,mav.uavAngEular,ma
print((mav1.uavPosNED,mav1.uavVelNED,mav1.uavAngEular
print((mav2.uavPosNED,mav2.uavVelNED,mav2.uavAngEular
print((mav3.uavPosNED,mav3.uavVelNED,mav3.uavAngEular
```

导入库

新建4个UDP连接

开始MAVLink数据监听

打印监听数据

```python
#开启Offboard模式
mav.initOffboard()
mav1.initOffboard()
mav2.initOffboard()
mav3.initOffboard()



#发送期望位置信号，飞到目标点 0,0，-1.7位置，偏航角为0
mav.SendPosNED(0, 0, -1.7, 0)
mav1.SendPosNED(2, 2, -1.7, 0)
mav2.SendPosNED(-2, -2, -1.7, 0)
mav3.SendPosNED(0, 0, -3, 0)


print("Send target Pos")
time.sleep(0.5)
#发送解锁命令
mav.SendMavArm(True)
time.sleep(2)
mav1.SendMavArm(True)
time.sleep(2)
mav2.SendMavArm(True)
time.sleep(2)
mav3.SendMavArm(True)
print("Send Arm Command")

time.sleep(10)
```

开启Offboard模式

发送期望位置

解锁，起飞

```python
#发送期望速度信号，0.2m/s向下降落，z轴向下为正
mav.SendVelNED(0, 0, 0.2, 0)
mav1.SendVelNED(0, 0, 0.2, 0)
mav2.SendVelNED(0, 0, 0.2, 0)
mav3.SendVelNED(0, 0, 0.2, 0)
print("Send Velocity Speed")

time.sleep(10)

#退出Offboard控制模式
print("Send offboard stop")
mav.endOffboard()
mav1.endOffboard()
mav2.endOffboard()
mav3.endOffboard()
time.sleep(1)

#退出Mavlink数据接收模式
print("Send Mavlink stop")
mav.stopRun()
mav1.stopRun()
mav2.stopRun()
mav3.stopRun()
time.sleep(1)
```

发送下降速度

等待降落

结束Offboard模式

结束MAVLink监听，仿真结束

4.进阶例子

4.9 Python控制集群无人机例程 – 运行效果

- “PythonSwarmFour”文件夹展示了用Python控制四个飞机的接口demo例程，其中“PX4MavCtrlV4Swarm.py”文件是关键的通讯接口模块
- 双击“PythonSwarmFour.bat”脚本可以启动四个飞机的软件在环（启用Mavlink数据协议），运行“PythonSwarmFour.py”可以使用接口获取四个飞机数据并控制其起飞下降。



谢谢！